LEARN SQL · PYTHON · DATA · AI

DROP TABLE;

Learn to code at a rare sneaker shop

- 29 coding shifts
- Python in every lesson
- Train your own AI model
- Real data · answer key



SQL PYTHON AI

Before you clock in

This book follows Big Face, a kid saving for one pair of sneakers, through 30 shifts at a rare-sneaker shop called the Grail Room. Every shift teaches one idea you can use for the rest of your life: how to ask data a question, how to write code, how to make a chart, how a machine learns.

How to use it

1. **Read a shift.** Each one has a story moment, the big idea, an example and a job for you.
2. **Try it for real.** Open the Drop Table page and work the same shift there. The page runs every query live, checks your answers and pays Big Face for them.
3. **Look it up.** Every Python puzzle starts with a short lesson. New words are in the Glossary at the back, with the day each one is taught.
4. **Check yourself.** Answers to every job and puzzle are in the Answer Key at the back. Try first!

What's real and what's practice

Table	Kind	What it holds	Dates
`releases`	REAL	Upcoming sneaker releases from the Sneaker Files release calendar	November 5, 2026 – December 31, 2026, checked November 5, 2026
`shoes`	PRACTICE	20 made-up shoes from four made-up brands	released March 7, 2026 – September 26, 2026
`sales`	PRACTICE	1,647 made-up consignment sales	August 1, 2026 – October 31, 2026

This book is a snapshot. The example results printed here use the release list as it stood on November 5, 2026; when a query says `date('now')`, the book treats "now" as that day. On the live page, the release list refreshes every morning, so your results there may differ. Release dates and prices can change; check the brand before you plan.

Meet the crew



Big Face

That's you. Real name Michael, but everybody calls him Big Face because he's always chasing big face hundreds: the $100 bills with the giant portrait. Saving up for the Halo Comet Mid "Gold Rush". Talks fast, learns faster.



Rae

The store manager. Runs the floor, the counter and the consignment office. Hands you a real job every shift in weeks 1 to 3.



Theo

The data lead. Lives in the back office with a laptop. Takes over in weeks 4 and 5 for Python, charts and machine learning.

Big Face's money

Big Face starts with **$60** of birthday money. Every shift pays wages: half the shift's rep points, in dollars. Get a SQL job right on the first try and Rae tips $10. In **The Flip** he can buy practice pairs and sell them later, minus a 10% app fee. And between shifts, stuff happens: 100 curveballs, good and bad. A few of them (and the math behind them) are in Appendix C.

Rep and ranks

Fresh Laces	Rotation Builder	Heat Spotter	Legit Checker	Grail Hunter	True Sneakerhead
0 rep	150 rep	400 rep	750 rep	1200 rep	1800 rep

The map: how a sneaker shop uses data

1. Collect: a public release calendar
2. Store: tables you query with SQL
3. Clean + chart: Python and pandas
4. Predict: machine learning
5. Explain: AI, checked by people

1

CHAPTER 1 · SQL BASICS

The stockroom



His real name is Michael, but nobody calls him that. Everybody calls him Big Face, because he's always chasing big face hundreds: the $100 bills with the giant portrait on the front. Today is his first day at the Grail Room, the rarest sneaker shop in Atlanta. He's just a kid saving for his first grail, and math class has never made sense. Rae hands him the release calendar: it's just a list, like the wishlist on his phone. That's data.

WHY THIS MATTERS TO YOU

Every app you use is tables with rows and columns: your playlists, your game inventory, your camera roll. SQL is how you ask them questions.

Day 1: What is data? · Day 2: Tables are shoe boxes · Day 3: Pick your columns · Day 4: Sort by price · Day 5: Filter by brand · Day 6: Search inside names

Day 1 9:00 AM · The stockroom · +30 rep · +$15 wages

What is data?



Rae
Welcome to the Grail Room! Before you touch a single box, you need to know what we track. Everything in this shop runs on data.

Big Face
Wait, my sneaker wishlist is data? Okay, this I get.



THE BIG IDEA

Data is facts you write down so you can look at them later. A shoe's price, its release day and its colors are all data.

LIKE YOU'RE 10 A sticker on a shoe box is data: size 9, style code, price. You can read it again tomorrow.

THE DETAIL

Each fact has a type: text, number, date, true/false, or missing. Analysis starts with knowing which one you have.

QUICK CHECK

1. Which one is data about a sneaker?

A. Go outside
B. Tomorrow is fun
C. I feel happy
D. Retail price: $130

2. A release date written as 2026-11-21 is...

A. a color
B. a date
C. a number you add up
D. a size

3. The price says 'not stated'. In data that's called...

A. missing (NULL)
B. an error you delete
C. zero
D. free

4. One shoe = one ___ in a table.

A. database
B. row
C. column
D. chart

PYTHON CORNER · LEARN IT FIRST · PRINT()

Python is a language for giving a computer instructions, one line at a time, top to bottom.

print() is how Python talks back. Whatever you put inside the round brackets shows up on the screen.

Text in Python goes inside quotes. The quotes are not printed; they just show Python where the text starts and stops. Text in quotes is called a string.

`print("Grail Room is open")`	Shows the words inside the quotes. No quotes on screen.
`print("Rae: be on time")`	Each print() starts a new line.
`print(7)`	Numbers don't need quotes.

OUTPUT

```
Grail Room is open
Rae: be on time
7
```

Remember: print(something) shows something. Quotes mark text; they don't print.

YOUR TURN · PRINT()

```
print("Big Face clocks in")
```

What shows up on the screen?

A. `Nothing`

B. `print Big Face clocks in`

C. `"Big Face clocks in"`

D. `Big Face clocks in`

Day 2 9:40 AM · The stockroom · +30 rep · +$15 wages

Tables are shoe boxes



Rae

This is the release wall: every shoe dropping soon, from a real calendar. Show me the first ten so I know you can read it.

Big Face

So a table is just a shoebox with labeled slots. Easy.



THE BIG IDEA

A table is like a wall of shoe boxes. Each box (row) is one shoe. Each label line (column) is one fact. SELECT means 'show me', and * means 'every column'.

LIKE YOU'RE 10 Pulling the first five boxes off the shelf to peek inside.

THE DETAIL

LIMIT caps how many rows come back. Handy when a table is big.

EXAMPLE

```
SELECT * FROM releases LIMIT 5;
```

WHAT IT RETURNS

id	release_date	brand	name	colorway	style_code	retail_usd
1	2026-11-05	Nike	Victor Wembanyama x Nike GT Cut 4 "Statue of Liberty"	Mint Foam/Metallic Gold	IW0737-300	210
2	2026-11-05	Nike	Nike Air Force 1 "What The LA"	White/Black-Hyper Jade	CT1117-100	125
3	2026-11-05	Nike	Nike Air Max 95 GORE-TEX "Black"	Black/Diffused Blue	IO1877-002	220
4	2026-11-05	Nike	Nike Air Force 1 Mid GTX Vibram "Black"	Black/Smoke Grey	IO1875-001	180
5	2026-11-05	Jordan	Jordan Son of Mars Low "Lunar Tunes"	Base Grey/Ghost-Stealth-White	IX6377-001	165

5 rows

YOUR JOB

Show the first 10 rows of releases.

```
SELECT * FROM releases LIMIT ___;
```

Hint: Change the number after LIMIT.

SAME JOB IN PYTHON (PANDAS)

```
import pandas as pd
releases = pd.read_csv('releases.csv')
releases.head(10)
```

PYTHON CORNER · LEARN IT FIRST · VARIABLES

A variable is a labeled box that holds a value. You make one with a single = sign: the name goes on the left, the value on the right.

Once the box exists, you use its name (no quotes) and Python swaps in what's inside.

Careful: print(shoes) shows what's in the box. print("shoes") with quotes just shows the word shoes.

`shoes = 3`	Make a box named shoes and put 3 in it.
`print(shoes)`	No quotes: Python looks inside the box and shows 3.
`print("shoes")`	Quotes: this is just text, so it shows the word.

OUTPUT

```
3
shoes
```

Remember: name = value stores it. Use the name without quotes to get the value back.

YOUR TURN · VARIABLES

```
wallet = 60
print(wallet)
```

What prints?

A. `60`

B. `wallet`

C. `An error`

D. `"60"`

Day 3 11:15 AM · The stockroom · +40 rep · +$20 wages

Pick your columns



Rae

A customer on the phone only wants names and prices. Don't read them the whole label!

Big Face

Only the columns I need. Less scrolling, more selling.



THE BIG IDEA

You don't always want every fact. Name the columns you want, with commas between them.

LIKE YOU'RE 10 Only reading the size and price off each box label.

EXAMPLE

```
SELECT name, release_date FROM releases LIMIT 5;
```

WHAT IT RETURNS

name	release_date
Victor Wembanyama x Nike GT Cut 4 "Statue of Liberty"	2026-11-05
Nike Air Force 1 "What The LA"	2026-11-05
Nike Air Max 95 GORE-TEX "Black"	2026-11-05
Nike Air Force 1 Mid GTX Vibram "Black"	2026-11-05
Jordan Son of Mars Low "Lunar Tunes"	2026-11-05

5 rows

YOUR JOB

Show the name and retail_usd of every release.

```
SELECT name, ___ FROM releases;
```

Hint: The price column is called retail_usd.

SAME JOB IN PYTHON (PANDAS)

```
releases[['name', 'retail_usd']]
```

PYTHON CORNER · LEARN IT FIRST · ADDING MONEY

Python does math with + - * and /. You can do math with variables too, because Python swaps in their values first.

The trick that confuses everyone: wallet = wallet + 10 is not a broken equation. Python works out the right side first (old wallet plus 10), then stores the answer back in wallet.

Numbers without quotes add up. Numbers inside quotes are text, and "5" + "5" glues them into "55".

`tips = 8`	tips starts at 8.
`tips = tips + 4`	Right side first: 8 + 4 = 12. Then 12 goes back into tips.
`print(tips)`	Shows the new value.

`print("5" + "5")`	These are strings, so + glues them together instead of adding.

OUTPUT

```
12
55
```

Remember: Right side runs first, then the answer goes into the box on the left.

YOUR TURN · ADDING MONEY

```
wallet = 60
wage = 15
wallet = wallet + wage
print(wallet)
```

Big Face gets paid. What prints?

A. 15

B. 60

C. 75

D. 6015

Day 4 1:30 PM · The stockroom · +40 rep · +$20 wages

Sort by price



Rae
We're making a 'Big Ticket' shelf. Find the five most expensive releases, priciest first.

Big Face
Big Ticket shelf? Say less. DESC, top five.



THE BIG IDEA

ORDER BY puts rows in order. Add DESC to start with the biggest number. LIMIT keeps just the top few.

LIKE YOU'RE 10 Lining up shoe boxes from most to least expensive.

THE DETAIL

In SQLite, empty prices (NULL) sort first when you go smallest-to-biggest.

EXAMPLE

```
SELECT name, retail_usd FROM releases ORDER BY retail_usd DESC LIMIT 3;
```

WHAT IT RETURNS

name	retail_usd
Air Jordan 4 "BIN 23"	355
Victor Solomon x Nike Air Foamposite One "Trophies Red"	350
Pharrell Williams x adidas Adistar Jellyfish "Dark Brown Frozen Green"	300

3 rows

YOUR JOB

Show the 5 most expensive releases: name and retail_usd, highest first.

```
SELECT name, retail_usd FROM releases ORDER BY retail_usd ___ LIMIT 5;
```

Hint: DESC means biggest first.

SAME JOB IN PYTHON (PANDAS)

```
releases.sort_values('retail_usd', ascending=False).head(5)[['name', 'retail_usd']]
```

PYTHON CORNER · LEARN IT FIRST · += SHORTCUT

Changing a box by a little bit happens so often that Python has shortcuts.

x += 5 means x = x + 5. x -= 3 means x = x − 3. There's also *= (multiply) and /= (divide).

Each line changes the box, and the next line starts from the new value.

`jar = 50`	The grail jar starts at 50.
`jar += 10`	Add 10: now 60.
`jar -= 25`	Take out 25: now 35.
`print(jar)`	Shows 35.

OUTPUT

```
35
```

Remember: += adds to what's already there. -= takes away.

YOUR TURN · += SHORTCUT

```
wallet = 75
wallet += 20
wallet -= 12
print(wallet)
```

He earns $20, then spends $12 on cleaner. What prints?

A. 83

B. 75

C. 63

D. 107

Day 5 3:05 PM · The stockroom · +50 rep · +$25 wages

Filter by brand



Rae

The adidas rep visits tomorrow. Pull every adidas release so I'm ready.

Big Face

Only adidas? WHERE brand equals 'adidas', all lowercase. Got it.



THE BIG IDEA

WHERE keeps only the rows you want. Text goes in single quotes, and spelling matters: 'adidas' is all lowercase in this data.

LIKE YOU'RE 10 Only pulling the boxes with the adidas logo.

EXAMPLE

```
SELECT name, release_date FROM releases WHERE brand = 'Vans';
```

WHAT IT RETURNS

name	release_date
Vans Old Skool 36 "Pearlized Marshmallow"	2026-11-12

1 row

YOUR JOB

Show name and release_date for every adidas release.

```
SELECT name, release_date FROM releases WHERE brand = '___';
```

Hint: The brand column holds the brand name.

SAME JOB IN PYTHON (PANDAS)

```
releases.loc[releases['brand'] == 'adidas', ['name', 'release_date']]
```

PYTHON CORNER · LEARN IT FIRST · COMPARING

Python can ask yes-or-no questions with comparison symbols: > (bigger), < (smaller), >= (bigger or equal), <= (smaller or equal), == (equal), != (not equal).

The answer is always True or False. These are called booleans, and they are capitalized in Python.

Don't mix up = and ==. One = stores a value. Two == asks a question.

`cash = 75`	Store 75.
`print(cash > 50)`	Is 75 bigger than 50? Yes.
`print(cash == 100)`	Is 75 equal to 100? No.
`print(cash != 100)`	Is 75 not equal to 100? Yes.

OUTPUT

```
True
False
True
```

Remember: Comparisons answer True or False. == asks; = stores.

YOUR TURN · COMPARING

```
price = 140
budget = 120
print(price <= budget)
```

Can he afford it?

A. `True`

B. `False`

C. `-20`

D. `20`

Day 6 5:20 PM · The stockroom · +50 rep · +$25 wages

Search inside names



Rae

Everyone keeps asking about the Space Jam stuff. Find every release with Space Jam in the name.

Big Face

Finding every Dunk without reading 111 names? Clutch.



THE BIG IDEA

LIKE searches for words inside text. % means 'anything can go here'. So '%Space Jam%' finds Space Jam anywhere in the name.

LIKE YOU'RE 10 Flipping through boxes for any label that says 'Space Jam'.

THE DETAIL

_ matches exactly one character. In SQLite, LIKE ignores upper/lower case for plain letters.

EXAMPLE

```
SELECT name FROM releases WHERE name LIKE '%Batman%';
```

WHAT IT RETURNS

name
Nike Air Max 90 "Batman"
Nike Air Max Plus "Batman"
Nike Air Max 95 Big Bubble "Batman"

3 rows

YOUR JOB

Show the name and retail_usd of every release with "Space Jam" in its name.

```
SELECT name, retail_usd FROM releases WHERE name LIKE '%___%';
```

Hint: Put % on both sides of the words.

SAME JOB IN PYTHON (PANDAS)

```
releases.loc[releases['name'].str.contains('Space Jam', case=False), ['name',
'retail_usd']]
```

PYTHON CORNER · LEARN IT FIRST · TEXT SEARCH

The word in checks whether something is inside something else. With text, it checks whether a small piece of text appears in a bigger one.

It's case-sensitive: "low" and "Low" are different. That's why people often do .lower() first, which turns all letters lowercase.

This is Python's version of SQL's LIKE '%...%'.

`shoe = "Comet Mid Gold"`	A string to search.
`print("Gold" in shoe)`	"Gold" is in there exactly. True.
`print("gold" in shoe)`	Lowercase g doesn't match capital G. False.
`print("gold" in shoe.lower())`	.lower() makes everything lowercase first, so now it matches.

OUTPUT

```
True
False
True
```

Remember: "piece" in text gives True or False. Watch the capital letters.

YOUR TURN · TEXT SEARCH

```
name = "Kitebird Flutter 2"
print("Flutter" in name)
```

What prints?

A. `False`

B. `True`

C. `Flutter`

D. 2



CHAPTER 2 · SQL: TIME + TOTALS

The front desk

Customers fire questions at the counter: when, how many, how much? Big Face figures out averages are the same math he uses to work out how many weeks until he can afford a pair.

WHY THIS MATTERS TO YOU

Saving up, planning a party, counting a streak: totals, averages and dates are math you already do in your head. Now you make a computer do it in one line.

Day 7: Pick a time period · Day 8: Missing prices · Day 9: Count and average · Day 10: Group by brand · Day 11: Drops per week · Day 12: Busy days (HAVING)

Day 7 10:00 AM · The front desk · +60 rep · +$30 wages

Pick a time period



Rae
A dad wants everything dropping Thanksgiving week. Which year? Always say which dates. Give me Nov 20 to Nov 30, 2026.

Big Face
This week only. Dates in quotes, year first. Noted.



THE BIG IDEA

Always say WHEN. Dates are written year-month-day, so they line up in order like words in a dictionary. BETWEEN includes both end days.

`LIKE YOU'RE 10` 'Shoes dropping Thanksgiving week' means nothing until you say which year.

EXAMPLE

```
SELECT release_date, name FROM releases
WHERE release_date BETWEEN '2026-12-01' AND '2026-12-07';
```

WHAT IT RETURNS

release_date	name
2026-12-01	Nike LeBron 24 "Chosen 1s"
2026-12-03	Nike Air Bakin "Black White"
2026-12-05	Pharrell Williams x adidas Adistar Jellyfish "Dark Brown Frozen Green"
2026-12-05	Pharrell x adidas Adistar Jellyfish "Auburn Timber"
2026-12-05	SoleFly x Air Jordan 5

6 rows, first 5 shown

YOUR JOB

Show release_date and name for releases from 2026-11-20 to 2026-11-30 (both days included), earliest first.

```
SELECT release_date, name FROM releases
WHERE release_date BETWEEN '___' AND '___'
ORDER BY ___;
```

Hint: Write dates as 'YYYY-MM-DD' in quotes.

SAME JOB IN PYTHON (PANDAS)

```
mask = releases['release_date'].between('2026-11-20', '2026-11-30')
releases.loc[mask, ['release_date', 'name']].sort_values('release_date')
```

PYTHON CORNER · LEARN IT FIRST · IF / ELSE

if lets Python make a decision. It checks a True/False question. If True, it runs the indented lines under it. If False, it skips them.

else is the backup plan: it runs only when the if question was False. Exactly one of the two runs, never both.

Two rules people forget: a colon : at the end of the if and else lines, and indenting (4 spaces) the lines that belong to each one.

`hype = 4`	Store 4.
`if hype >= 8:`	Is 4 >= 8? No, so skip the indented line under it.
`    print("Put it up front")`	(Skipped.)
`else:`	The if was False, so the else part runs.
`    print("Back shelf")`	Shows Back shelf.
`print("Done")`	Not indented, so it always runs after the decision.

OUTPUT

```
Back shelf
Done
```

Remember: if True → run the indented block. Otherwise → run else. Colons and indents matter.

YOUR TURN · IF / ELSE

```
wallet = 90
price = 110
if wallet >= price:
    print("Buy it")
else:
    print("Keep saving")
```

What does Big Face do?

A. Both lines

B. Nothing

C. Keep saving

D. Buy it

Day 8 11:30 AM · The front desk · +50 rep · +$25 wages

Missing prices



Rae

Some price tags on the wall are blank. List the shoes without a price so I can chase the brands.

Big Face

No price yet means NULL, not zero. Sneaky.



THE BIG IDEA

Some shoes don't have a price yet. Empty in SQL is called NULL. You can't use = NULL; you have to write IS NULL.

LIKE YOU'RE 10 A box with a blank price sticker. It isn't free; nobody wrote the price yet.

THE DETAIL

NULL means unknown. AVG and other totals skip NULLs, so COUNT(*) and COUNT(retail_usd) can be different.

EXAMPLE

```
SELECT name FROM releases WHERE colorway IS NULL;
```

WHAT IT RETURNS

name
adidas Anthony Edwards 3 "Purple Iridescent"
Cactus Jack x Rockstar Games x Nike Air Force 1 Low "GTA 6" Pack
Oregon Ducks Nike Air Force 1 Low
UNheardof x adidas Adistar Control 5 "Doggy Runner"
Social Status x Air Jordan 1 Low OG

9 rows, first 5 shown

YOUR JOB

Show release_date and name for releases with no price listed.

```
SELECT release_date, name FROM releases WHERE ___;
```

Hint: Use IS NULL, not = NULL.

SAME JOB IN PYTHON (PANDAS)

```
releases.loc[releases['retail_usd'].isna(), ['release_date', 'name']]
```

PYTHON CORNER · LEARN IT FIRST · NONE (MISSING)

Sometimes a value is missing: no price yet, no release date yet. Python uses None for 'nothing here'. It's the same idea as NULL in SQL.

None is not 0 and it's not empty text. It means unknown.

To check for it, write is None (or is not None). That's the Python habit, just like IS NULL in SQL.

`resale = None`	Nothing known yet.
`print(resale)`	Printing it shows the word None.
`if resale is None:`	The check for missing values.
`    print("No resale price yet")`	Runs because resale really is None.

OUTPUT

```
None
No resale price yet
```

Remember: None means missing. Check it with is None.

YOUR TURN · NONE (MISSING)

```
retail = None
if retail is None:
    print("Price TBA")
```

What prints?

A. `0`

B. `None`

C. `Nothing`

D. `Price TBA`

Day 9 1:00 PM · The front desk · +60 rep · +$30 wages

Count and average



Rae
The owner texts: 'How many drops are on the wall, and what's a typical price?' One answer, one row.

Big Face
Average price is the same math as my savings plan!



THE BIG IDEA

COUNT counts rows. AVG finds the average (add them up, divide by how many). ROUND makes it tidy. AS gives the answer a name.

LIKE YOU'RE 10 Counting every box in the stack, then working out the typical price.

EXAMPLE

```
SELECT COUNT(*) AS releases, MIN(retail_usd) AS cheapest, MAX(retail_usd) AS priciest
FROM releases;
```

WHAT IT RETURNS

releases	cheapest	priciest
111	120	355

1 row

YOUR JOB

One row: how many releases there are (call it releases) and the average retail price rounded to whole dollars (call it avg_price).

```
SELECT ___ AS releases, ROUND(___) AS avg_price FROM releases;
```

Hint: COUNT(*) counts rows.

SAME JOB IN PYTHON (PANDAS)

```
pd.Series({'releases': len(releases), 'avg_price':
round(releases['retail_usd'].mean())})
```

PYTHON CORNER · LEARN IT FIRST · PERCENT MATH

Percents in Python are written as decimals: 10% is 0.10, 25% is 0.25, 7% is 0.07.

To take a percent off, multiply by (1 − percent). 1 − 0.25 = 0.75, so 25% off means 'keep 75%'.

When decimals are involved, Python shows the answer as a decimal too, so 100 becomes 100.0. Same value, it just has a .0.

`price = 80`	Full price.
`discount = 0.25`	25% written as a decimal.
`sale = price * (1 - discount)`	Brackets first: 1 − 0.25 = 0.75. Then 80 × 0.75.
`print(sale)`	Shows 60.0 (decimal math gives a decimal answer).

OUTPUT

```
60.0
```

Remember: x% off → price * (1 - x/100). Decimal in, decimal out.

YOUR TURN · PERCENT MATH

```
price = 200
fee = 0.10
you_get = price * (1 - fee)
print(you_get)
```

He sells a pair for $200 with a 10% app fee. What prints?

A. `20.0`

B. `200`

C. `180.0`

D. `190.0`

Day 10 2:45 PM · The front desk · +70 rep · +$35 wages

Group by brand



Rae

How much wall space does each brand need? Count each brand's drops and their average price.

Big Face
One row per brand. That's a whole report in one line.



THE BIG IDEA

GROUP BY makes piles. Put every Nike in one pile, every Jordan in another, then count each pile.

LIKE YOU'RE 10 Sorting the boxes into piles by logo, then counting each pile.

THE DETAIL

One output row per group. Every column you SELECT must be in the GROUP BY or inside a total like COUNT or AVG.

EXAMPLE

```
SELECT brand, COUNT(*) AS drops FROM releases GROUP BY brand;
```

WHAT IT RETURNS

brand	drops
Jordan	34
Nike	72
Vans	1
adidas	4

4 rows

YOUR JOB

For each brand: brand, number of releases (drops) and average price rounded (avg_price). Most drops first.

```
SELECT brand, ___ AS drops, ___ AS avg_price
FROM releases
GROUP BY ___
ORDER BY ___;
```

Hint: GROUP BY brand makes one row per brand.

SAME JOB IN PYTHON (PANDAS)

```
(releases.groupby('brand')
  .agg(drops=('name', 'size'), avg_price=('retail_usd', 'mean'))
  .round().sort_values('drops', ascending=False))
```

PYTHON CORNER · LEARN IT FIRST · F-STRINGS

You often want to mix text and numbers in one message. An f-string does that.

Put an f right before the opening quote. Then anything inside {curly brackets} gets replaced by its value.

Everything outside the curly brackets prints exactly as written, including symbols like $.

`name = "Theo"`	A text variable.
`pairs = 12`	A number variable.
`print(f"{name} sold {pairs} pairs")`	f-string: the {} get filled in.
`print("{name} sold {pairs} pairs")`	No f, so the curly brackets print as-is.

OUTPUT

```
Theo sold 12 pairs
{name} sold {pairs} pairs
```

Remember: f"...{var}..." drops the value into the text. No f, no fill-in.

YOUR TURN · F-STRINGS

```
shoe = "Dash 5"
profit = 35
print(f"Flipped the {shoe} for +${profit}")
```

What prints?

A. `An error`

B. `f"Flipped the Dash 5 for +$35"`

C. `Flipped the Dash 5 for +$35`

D. `Flipped the {shoe} for +${profit}`

Day 11 4:10 PM · The front desk · +70 rep · +$35 wages

Drops per week



Rae

Which weeks will be crazy busy? I need to schedule extra staff. Count drops per week, then press Chart.

Big Face

Drops per week... so THAT'S why Saturdays are packed.



THE BIG IDEA

strftime pulls pieces out of a date. '%Y-W%W' turns a date into a year and week number, so you can count drops each week.

LIKE YOU'RE 10 Putting the boxes into one bin per week, then seeing which bin is fullest.

THE DETAIL

'%Y-%m' gives the month. Turning dates into buckets is how you build a time series.

EXAMPLE

```
SELECT strftime('%Y-%m', release_date) AS month, COUNT(*) AS drops
FROM releases GROUP BY month ORDER BY month;
```

WHAT IT RETURNS

month	drops
2026-11	82
2026-12	29

2 rows

YOUR JOB

Count releases per week. Columns: week (use strftime('%Y-W%W', release_date)) and drops. Oldest week first.

```
SELECT strftime('___', release_date) AS week, COUNT(*) AS drops
FROM releases GROUP BY ___ ORDER BY ___;
```

Hint: Group by the week you made: GROUP BY week.

SAME JOB IN PYTHON (PANDAS)

```
dates = pd.to_datetime(releases['release_date'])
releases.groupby(dates.dt.strftime('%Y-W%W')).size()
```

PYTHON CORNER · LEARN IT FIRST · ROUND()

Math often gives long decimals like 183.6667. round() tidies them up.

round(x, 2) keeps 2 digits after the decimal point. round(x, 1) keeps 1. round(x) with no second number gives a whole number.

It looks at the next digit to decide: 5 or more rounds up, less rounds down.

`avg = 47.6382`	A messy average.
`print(round(avg, 2))`	Two decimals: 47.6382 → 47.64 (the 8 rounds the 3 up).
`print(round(avg, 1))`	One decimal: 47.6.
`print(round(avg))`	No decimals: 48.

OUTPUT

```
47.64
47.6
48
```

Remember: round(x, n) keeps n decimal places.

YOUR TURN · ROUND()

```
avg = 183.6667
print(round(avg, 1))
```

What prints?

A. `183.66`

B. `184`

C. `183.6`

D. `183.7`

Day 12 6:00 PM · The front desk · +70 rep · +$35 wages

Busy days (HAVING)



Rae
Line-around-the-block days: which dates have 5 or more drops? Those days we open early.

Big Face
Busy days only. HAVING filters the groups. Smart.



THE BIG IDEA

WHERE filters boxes. HAVING filters piles, after you GROUP BY.

LIKE YOU'RE 10 After making a pile for each day, only keep the tall piles.

THE DETAIL

WHERE runs before grouping; HAVING runs after. You can't use COUNT() inside WHERE.

EXAMPLE

```
SELECT brand, COUNT(*) AS drops FROM releases GROUP BY brand HAVING COUNT(*) > 10;
```

WHAT IT RETURNS

brand	drops
Jordan	34
Nike	72

2 rows

YOUR JOB

Show release_date and drops for days with 5 or more releases, busiest first.

```
SELECT release_date, COUNT(*) AS drops FROM releases
GROUP BY ___ HAVING ___ ORDER BY ___;
```

Hint: HAVING comes after GROUP BY.

SAME JOB IN PYTHON (PANDAS)

```
per_day = releases.groupby('release_date').size()
per_day[per_day >= 5].sort_values(ascending=False)
```

PYTHON CORNER · LEARN IT FIRST · INTEGER DIVISION

Normal division / always gives a decimal: 7 / 2 is 3.5.

Floor division // gives only the whole number part and throws away the rest: 7 // 2 is 3. Think 'how many full ones fit?'

Modulo % gives the leftover: 7 % 2 is 1. Think 'what's left over after the full ones?'

`cash = 50`	Big Face has $50 and laces cost $15.
`print(cash / 15)`	Regular division: a long decimal.
`print(cash // 15)`	How many full $15 laces fit? 3.
`print(cash % 15)`	Leftover after 3 × 15 = 45: $5.

OUTPUT

```
3.3333333333333335
3
5
```

Remember: // = how many fit. % = what's left over.

YOUR TURN · INTEGER DIVISION

```
wallet = 95
price = 30
print(wallet // price, wallet % price)
```

How many $30 raffle tickets, and how much is left?

A. `3 0`

B. `3 5`

C. `65 30`

D. `3.17 0`



CHAPTER 3 · SQL: JOINS + LOGIC

The consignment office

Consigners bring pairs to sell. Big Face links sales to shoes and works out which ones sell for way over retail: percentages, finally useful.

WHY THIS MATTERS TO YOU

Your bank app joins purchases to store names. Your game joins players to scores. Joins are how separate lists turn into answers.

Day 13: Meet the sales log · Day 14: JOIN two tables · Day 15: Heat or brick? (CASE) · Day 16: Beat the average (WITH) · Day 17: Rank the brands

Day 13 10:00 AM · The consignment office · +60 rep · +$30 wages

Meet the sales log



Rae

Promotion! You're in consignment now. People bring us shoes, we sell them, we keep a cut. Here's our sales log. It's PRACTICE data with made-up brands, Aug 1 to Oct 31, 2026. How many pairs did we sell in September?

Big Face

Fake brands, real lessons. Let's see who sells.



THE BIG IDEA

shoes has one row per shoe we carry. sales has one row per pair sold, with the date, size and price.

LIKE YOU'RE 10 One notebook lists every shoe. Another notebook lists every time someone bought a pair.

THE DETAIL

shoes is a 'dimension' table (things); sales is a 'fact' table (events). Facts point at things with shoe_id.

EXAMPLE

```
SELECT * FROM shoes LIMIT 5;
-- then try: SELECT * FROM sales LIMIT 5;
```

WHAT IT RETURNS

shoe_id	brand	model	colorway	release_date	retail_usd	pairs_made	hype
1	Northbound	Glide 1	Ice Blue	2026-03-14	120	40000	3
2	Northbound	Glide 1	Lava	2026-05-02	120	15000	7
3	Northbound	Summit Hi	Forest	2026-04-11	160	25000	5
4	Northbound	Summit Hi	Midnight	2026-07-18	160	8000	8
5	Northbound	Trail Kid	Sunrise	2026-06-06	90	60000	2

5 rows

YOUR JOB

How many sales happened in September 2026? One column: sales.

```
SELECT COUNT(*) AS sales FROM sales WHERE ___;
```

Hint: September has 30 days.

SAME JOB IN PYTHON (PANDAS)

```
sales = pd.read_csv('sales.csv')
sales['sale_date'].between('2026-09-01', '2026-09-30').sum()
```

PYTHON CORNER · LEARN IT FIRST · LISTS

A list holds many values in order, inside square brackets, separated by commas.

You grab one item with its position number in square brackets. Python starts counting at 0, so the first item is [0], the second is [1].

Negative numbers count from the end: [-1] is the last item, [-2] is second to last.

`sizes = [8, 9, 10, 11]`	A list of 4 shoe sizes. Positions are 0, 1, 2, 3.
`print(sizes[0])`	Position 0 is the first item: 8.
`print(sizes[2])`	Position 2 is the third item: 10.
`print(sizes[-1])`	-1 is the last item: 11.

OUTPUT

```
8
10
11
```

Remember: Lists count from 0. [-1] is the last one.

YOUR TURN · LISTS

```
prices = [120, 95, 210, 150]
print(prices[0], prices[-1])
```

What prints?

A. `1 4`

B. `120 210`

C. `120 150`

D. `95 150`

Day 14 11:45 AM · The consignment office · +90 rep · +$45 wages

JOIN two tables



Rae
The sales log only says shoe_id 10, shoe_id 17… nobody knows what that is. Glue the names on and show each shoe's sales and average price.

Big Face
shoe_id 10 is the Comet Mid. My grail. Of course it is.



THE BIG IDEA

sales only knows a shoe_id number. JOIN glues each sale to its shoe so you can see the name. ON says which column matches.

`LIKE YOU'RE 10` Matching each receipt to the shoe box with the same number.

THE DETAIL

INNER JOIN keeps rows where the key matches on both sides. Short aliases (s, sh) keep it readable. Ask: what's the key, and is it one-to-many?

EXAMPLE

```
SELECT s.sale_date, sh.model, sh.colorway, s.price_usd
FROM sales s JOIN shoes sh ON s.shoe_id = sh.shoe_id
LIMIT 5;
```

WHAT IT RETURNS

sale_date	model	colorway	price_usd
2026-08-01	Glide 1	Lava	104
2026-08-01	Glide 1	Lava	93
2026-08-01	Summit Hi	Midnight	225
2026-08-01	Summit Hi	Midnight	257
2026-08-01	Summit Hi	Midnight	249

5 rows

YOUR JOB

For each shoe: model, colorway, number of sales (sales) and average resale price rounded (avg_price). Most sales first.

```
SELECT sh.model, sh.colorway, ___ AS sales, ___ AS avg_price
FROM sales s JOIN shoes sh ON ___
GROUP BY ___ ORDER BY ___;
```

Hint: The matching column is shoe_id in both tables.

SAME JOB IN PYTHON (PANDAS)

```
df = sales.merge(shoes, on='shoe_id')
(df.groupby(['shoe_id', 'model', 'colorway'])
  .agg(sales=('sale_id', 'size'), avg_price=('price_usd', 'mean'))
  .round().sort_values('sales', ascending=False))
```

PYTHON CORNER · LEARN IT FIRST · LEN() AND SUM()

Python has built-in helpers for lists of numbers. len() counts the items. sum() adds them up.

An average is the total divided by how many there are, so sum(x) / len(x).

That's exactly what SQL's COUNT(), SUM() and AVG() do.

`tips = [5, 10, 15]`	Three tips.
`print(len(tips))`	How many items? 3.
`print(sum(tips))`	Total: 5 + 10 + 15 = 30.
`print(sum(tips) / len(tips))`	Average: 30 ÷ 3 = 10.0 (division always gives a decimal).

OUTPUT

```
3
30
10.0
```

Remember: len = how many, sum = total, sum/len = average.

YOUR TURN · LEN() AND SUM()

```
sales = [200, 180, 240, 220]
print(sum(sales) / len(sales))
```

What's the average sale?

A. 4

B. 840

C. 210.0 D. 220

Day 15 2:00 PM · The consignment office · +90 rep · +$45 wages

Heat or brick? (CASE)



Rae

Consigners ask: 'Is my shoe heat or a brick?' Work out each shoe's premium and stamp it HEAT, FAIR or BRICK.

Big Face

Heat or brick, I'm labeling everything now.



THE BIG IDEA

Premium = resale price minus store price. Positive means it sells for more than the store price (heat). Negative means less (brick). CASE lets you write your own labels.

`LIKE YOU'RE 10` If the shoe sells for more than the store price, stamp it HEAT. Less, stamp it BRICK.

THE DETAIL

Use premium as a percent of retail so cheap and expensive shoes compare fairly. CASE WHEN … THEN … ELSE … END.

EXAMPLE

```
SELECT model, colorway, retail_usd,
  CASE WHEN hype >= 8 THEN 'hyped' ELSE 'chill' END AS vibe
FROM shoes;
```

WHAT IT RETURNS

model	colorway	retail_usd	vibe
Glide 1	Ice Blue	120	chill
Glide 1	Lava	120	chill
Summit Hi	Forest	160	chill
Summit Hi	Midnight	160	hyped
Trail Kid	Sunrise	90	chill

20 rows, first 5 shown

YOUR JOB

For each shoe: model, colorway, premium_pct = ROUND(100.0 * (AVG(price_usd) - retail_usd) / retail_usd) and label: 'HEAT' if premium_pct >= 20, 'FAIR' if >= 0, else 'BRICK'. Highest premium first.

```
SELECT sh.model, sh.colorway,
  ROUND(100.0 * (AVG(s.price_usd) - sh.retail_usd) / sh.retail_usd) AS
premium_pct,
  CASE WHEN ___ THEN 'HEAT' WHEN ___ THEN 'FAIR' ELSE 'BRICK' END AS label
FROM sales s JOIN shoes sh ON s.shoe_id = sh.shoe_id
GROUP BY ___ ORDER BY ___;
```

Hint: Repeat the premium formula inside CASE (SQLite also lets you reuse the alias).

SAME JOB IN PYTHON (PANDAS)

```
g = df.groupby(['shoe_id', 'model', 'colorway', 'retail_usd'])
['price_usd'].mean().reset_index()
g['premium_pct'] = (100 * (g['price_usd'] - g['retail_usd']) /
g['retail_usd']).round()
g['label'] = pd.cut(g['premium_pct'], [-1e9, -0.5, 19.5, 1e9], labels=['BRICK',
'FAIR', 'HEAT'])
```

PYTHON CORNER · LEARN IT FIRST · FOR LOOPS

A for loop repeats the same lines once for each item in a list.

for item in list: picks the first item, puts it in the variable, runs the indented lines, then moves to the next item, until the list runs out.

A common pattern: start a total at 0 before the loop, then add to it inside the loop.

`total = 0`	Start the total at zero, before the loop.
`for p in [10, 20, 5]:`	Loop over the list. p will be 10, then 20, then 5.
`    total += p`	Add this item to the total.

`    print("added", p, "total is", total)`	Shows progress on each turn of the loop.
`print("final", total)`	Not indented: runs once, after the loop finishes.

OUTPUT

```
added 10 total is 10
added 20 total is 30
added 5 total is 35
final 35
```

Remember: for x in list: runs the indented lines once per item.

YOUR TURN · FOR LOOPS

```
wallet = 0
for wage in [20, 35, 25]:
    wallet += wage
print(wallet)
```

Three shifts of wages. What prints?

A. `25`

B. `[20, 35, 25]`

C. `3`

D. `80`

Day 16 3:30 PM · The consignment office · +100 rep · +$50 wages

Beat the average (WITH)



Rae
Which shoes sell for more than our average sale? Those get the glass case by the door.

Big Face
Beat the average? I do that every shift.



THE BIG IDEA

WITH lets you make a mini-table first, give it a name, then use it. Work out each shoe's average, then keep the ones above the store-wide average.

LIKE YOU'RE 10 First work out the class average, then find who scored above it.

THE DETAIL

CTEs (WITH name AS (...)) break a problem into named steps. A subquery like (SELECT AVG(...) FROM ...) returns one number you can compare against.

EXAMPLE

```
WITH per_shoe AS (
  SELECT shoe_id, AVG(price_usd) AS avg_price FROM sales GROUP BY shoe_id
)
SELECT * FROM per_shoe ORDER BY avg_price DESC LIMIT 3;
```

WHAT IT RETURNS

shoe_id	avg_price
10	362.43
20	319.15
4	189.52

3 rows

YOUR JOB

Using WITH, list model, colorway and avg_price (rounded) for shoes whose average sale price is above the average of ALL sales. Highest first.

```
WITH per_shoe AS (
  SELECT shoe_id, AVG(price_usd) AS avg_price FROM sales GROUP BY ___
)
SELECT ___
FROM per_shoe p JOIN shoes sh ON ___
WHERE p.avg_price > (___)
ORDER BY ___;
```

Hint: The overall average is (SELECT AVG(price_usd) FROM sales).

SAME JOB IN PYTHON (PANDAS)

```
per_shoe = sales.groupby('shoe_id')['price_usd'].mean()
above = per_shoe[per_shoe > sales['price_usd'].mean()]
shoes.set_index('shoe_id').loc[above.index, ['model',
'colorway']].assign(avg_price=above.round())
```

PYTHON CORNER · LEARN IT FIRST · LOOP + IF

You can put an if inside a for loop. The loop visits every item, and the if decides which items get acted on.

Watch the indents: the if is indented once (inside the loop), and the line under the if is indented twice (inside the if).

This is how WHERE works in SQL: look at every row, keep the ones that pass the test.

`sizes = [9, 12, 10, 13]`	Four sizes.
`for s in sizes:`	Visit each size.
`    if s >= 12:`	Test it.
`        print("big size", s)`	Only runs when the test is True: for 12 and 13.

OUTPUT

```
big size 12
big size 13
```

Remember: Loop visits everything; the if inside picks what to act on.

YOUR TURN · LOOP + IF

```
prices = [90, 310, 150, 400]
for p in prices:
    if p > 300:
        print("heat", p)
```

What prints?

A. `heat 310`

B. `heat 310 then heat 400`

C. `heat 400`

D. `All four prices`

Day 17 5:45 PM · The consignment office · +120 rep · +$60 wages

Rank the brands



Rae

End of the quarter. Rank the brands by how much money they made us. Medal ceremony!

Big Face
Ranking brands like a sneaker tier list. Love it.



THE BIG IDEA

Window functions look at the whole table without squishing rows together. RANK() hands out 1st, 2nd, 3rd place.

LIKE YOU'RE 10 Handing out medals while everyone stays in line.

THE DETAIL

RANK() OVER (ORDER BY ...) ranks; SUM(...) OVER (PARTITION BY ... ORDER BY ...) makes running totals; AVG over ROWS BETWEEN 6 PRECEDING AND CURRENT ROW is a 7-row moving average.

EXAMPLE

```
SELECT sale_date, price_usd,
  AVG(price_usd) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
moving_avg
FROM sales WHERE shoe_id = 10 LIMIT 15;
```

WHAT IT RETURNS

sale_date	price_usd	moving_avg
2026-09-12	434	434
2026-09-13	419	426.5
2026-09-13	435	429.33
2026-09-13	421	427.25
2026-09-13	399	421.6

15 rows, first 5 shown

YOUR JOB

Rank brands by total sales money in the period. Columns: brand, revenue (SUM of price_usd), place (RANK by revenue, highest = 1). Order by place.

```
SELECT sh.brand, SUM(s.price_usd) AS revenue,
  RANK() OVER (___) AS place
FROM sales s JOIN shoes sh ON ___
GROUP BY ___ ORDER BY ___;
```

Hint: Highest revenue gets place 1, so DESC.

SAME JOB IN PYTHON (PANDAS)

```
rev = df.groupby('brand')['price_usd'].sum().sort_values(ascending=False)
rev.to_frame('revenue').assign(place=rev.rank(ascending=False, method='min'))
```

PYTHON CORNER · LEARN IT FIRST · FUNCTIONS

A function is a reusable recipe. You write it once with def, then call it whenever you need it.

The names in the brackets are parameters: blanks you fill in when you call it. return sends the answer back.

A parameter can have a default, like tax=0.07. If you don't pass that one, the default is used.

`def total(price, tax=0.07):`	Make a recipe called total. tax has a default of 0.07.
`    return round(price * (1 + tax), 2)`	Do the math and send the answer back.
`print(total(100))`	No tax given, so the default 7% is used: 107.0.
`print(total(100, 0.10))`	Tax given as 0.10, so 10% is used: 110.0.

OUTPUT

```
107.0
110.0
```

Remember: def makes a recipe, return hands back the answer, defaults fill in blanks you skip.

YOUR TURN · FUNCTIONS

```
def profit(sold, paid, fee=0.10):
    return round(sold * (1 - fee) - paid)

print(profit(250, 160))
```

He paid $160 and sold for $250. What's his profit after the fee?

A. 65

B. 75

C. -65

D. 90



CHAPTER 4 · PYTHON + ANALYTICS

The back office

Theo pulls Big Face into the back office and opens a laptop: Python. Same questions as before, but code can loop through thousands of rows while you grab lunch.

WHY THIS MATTERS TO YOU

Python is one of the most popular languages for beginners and for data jobs. Scientists, game studios and streaming apps use it every day.

Day 18: Python: lists and loops · Day 19: Python: dictionaries and functions · Day 20: pandas: SQL's twin · Day 21: How spread out? (analytics) · Day 22: Clean it, chart it (notebook)

Day 18 10:00 AM · The back office · +70 rep · +$35 wages

Python: lists and loops



Theo

I'm Theo, the data lead. Upstairs we use Python too: it does everything SQL does, plus cleaning, charts and machine learning. Let's start simple. Read each snippet and say what it prints.

Big Face

Python?! Okay Theo, show me the loops.



THE BIG IDEA

A list holds many values in order, like a stack of price tags. A for loop visits each one. Python counts positions from 0.

LIKE YOU'RE 10 A list is a row of shoe boxes. A loop is walking past each box and reading its tag.

THE DETAIL

Run every snippet yourself in the notebook kit (Back office → Download the kit), lesson 01.

QUICK CHECK

1. What prints?

```
prices = [130, 215, 255]
print(prices[0])
```

A. an error

B. 215

C. 130

D. [130]

2. What prints?

```
prices = [130, 215, 255]
print(len(prices))
```

A. 3

B. 600

C. 2

D. 255

3. What prints?

```
total = 0
for p in [130, 215, 255]:
    total = total + p
print(total)
```

A. 130215255

B. 255

C. 0

D. 600

4. What prints?

```
for p in [130, 215, 255]:
    if p > 200:
        print('heat', p)
```

A. heat 215, then heat 255

B. only heat 255

C. heat 130, heat 215 and heat 255

D. nothing

5. What prints?

```
print([p * 2 for p in [10, 20]])
```

A. [20, 40]

B. [10, 20, 10, 20]

C. [10, 20]

D. 60

PYTHON CORNER · LEARN IT FIRST · LISTS GROW

Lists can change. .append(x) adds x to the end of the list.

len() tells you how many items are in it right now, so it grows as you append.

Positions still start at 0. After appending, the new item is at the end ([-1]).

`cart = []`	An empty list.
`cart.append("laces")`	Add one item: ["laces"].
`cart.append("cleaner")`	Add another to the end.
`print(cart)`	Shows the whole list.
`print(len(cart), cart[-1])`	2 items, and the last one is cleaner.

OUTPUT

```
['laces', 'cleaner']
2 cleaner
```

Remember: .append() adds to the end. len() counts what's there now.

YOUR TURN · LISTS GROW

```
closet = ["Glide 1"]
closet.append("Orbit 3")
closet.append("Dash 5")
print(len(closet), closet[1])
```

What prints?

A. `3 Orbit 3`

B. `2 Orbit 3`

C. `3 Dash 5`

D. `3 Glide 1`

Day 19 11:30 AM · The back office · +70 rep · +$35 wages

Python: dictionaries and functions



Theo
A dictionary is how Python stores a shoe's label. A function is a recipe you can reuse. These two run half our scripts.

Big Face
A dictionary is a pair with labels. Model, price, hype.



THE BIG IDEA

A dictionary matches names to values, like 'price' → 230. A function takes inputs and gives back an answer with return.

LIKE YOU'RE 10 A dictionary is a shoe box label: each line has a name and a value. A function is a calculator button you made yourself.

THE DETAIL

Notebook kit lesson 01 has every snippet ready to run and change.

QUICK CHECK

1. What prints?

```
shoe = {'name': 'Air Jordan 4', 'price': 230}
print(shoe['price'])
```

A. Air Jordan 4

B. an error

C. price

D. 230

2. What prints?

```
def premium(resale, retail):
    return resale - retail

print(premium(300, 230))
```

A. 70

B. 530

C. -70

D. premium

3. What prints?

```
shoe = {'name': 'Air Jordan 4', 'price': 230}
shoe['size'] = 10
print(len(shoe))
```

A. 2

B. 3

C. an error

D. 10

4. What prints?

```
stock = {'AJ4': 3, 'Kobe 8': 0}
print([k for k, v in stock.items() if v == 0])
```

A. {'Kobe 8': 0}

B. ['Kobe 8']

C. [0]

D. ['AJ4']

5. What prints?

```
print(round(2 / 3, 2))
```

A. 0.6666

B. 0.66

C. 1

D. 0.67

PYTHON CORNER · LEARN IT FIRST · DICTIONARIES

A dictionary (dict) stores values under labels called keys. It's like one row from a table: each column name points to a value.

Curly brackets make one: {"key": value, ...}. You read a value with square brackets and the key in quotes.

Setting a key that already exists replaces the old value. Setting a new key adds it.

`shoe = {"model": "Glide 1", "size": 10}`	A dict with two keys: model and size.
`print(shoe["model"])`	Look up the value under model.
`shoe["size"] = 11`	size exists already, so 10 is replaced with 11.
`shoe["price"] = 140`	price is new, so it gets added.
`print(shoe)`	Shows the whole dict.

OUTPUT

```
Glide 1
{'model': 'Glide 1', 'size': 11, 'price': 140}
```

Remember: dict["key"] reads a value. dict["key"] = x sets it.

YOUR TURN · DICTIONARIES

```
pair = {"model": "Comet Mid", "paid": 410, "hype": 10}
pair["paid"] = 395
print(pair["model"], pair["paid"])
```

What prints?

A. `Comet Mid 410`

B. `Comet Mid 395`

C. `model paid`

D. `An error`

Day 20 1:15 PM · The back office · +80 rep · +$40 wages

pandas: SQL's twin



Theo

Everything you did downstairs in SQL has a pandas version. Match each SQL query to the pandas code that does the same thing.

Big Face

pandas is SQL in a different jersey.



THE BIG IDEA

pandas is a Python library for tables. A table in pandas is called a DataFrame. Same ideas as SQL, different words.

LIKE YOU'RE 10 SQL and pandas are two languages that say the same thing, like 'hello' and 'hola'.

THE DETAIL

Notebook kit lesson 02 runs every pair side by side on the real release data.

QUICK CHECK

1. Which pandas line does the same?

```
SELECT name, retail_usd FROM releases;
```

A. releases.name.retail_usd

B. releases[['name', 'retail_usd']]

C. releases.select('name')

D. releases.head()

2. Which pandas line does the same?

```
SELECT * FROM releases WHERE brand = 'Nike';
```

A. releases.brand = 'Nike'

B. releases.where('Nike')

C. releases['Nike']

D. releases[releases['brand'] == 'Nike']

3. Which pandas line does the same?

```
SELECT * FROM releases ORDER BY retail_usd DESC;
```

A. releases.sort_values('retail_usd', ascending=False)

B. releases.order_by('retail_usd')

C. releases.max('retail_usd')

D. releases.sort('retail_usd')

4. Which pandas line does the same?

```
SELECT brand, COUNT(*) FROM releases GROUP BY brand;
```

A. releases.groupby('brand').size()

B. releases.group('brand')

C. releases.count('brand')

D. releases['brand'].sum()

5. Which pandas line does the same?

```
SELECT * FROM sales s JOIN shoes sh ON s.shoe_id = sh.shoe_id;
```

A. sales.join('shoes')

B. pd.concat([sales, shoes])

C. sales + shoes

D. sales.merge(shoes, on='shoe_id')

PYTHON CORNER · LEARN IT FIRST · LIST COMPREHENSION

A list comprehension builds a new list from an old one in a single line.

Read [p * 2 for p in prices] as: 'p times 2, for every p in prices'. It's a for loop squeezed into square brackets.

You can add an if at the end to keep only some items: [p for p in prices if p > 100].

`prices = [50, 120, 200]`	The starting list.
`doubled = [p * 2 for p in prices]`	Make a new list: each price × 2.
`big = [p for p in prices if p > 100]`	Make a new list with only prices over 100.
`print(doubled)`	[100, 240, 400]
`print(big)`	[120, 200]

OUTPUT

```
[100, 240, 400]
[120, 200]
```

Remember: [do_this for x in list if test] builds a new list in one line.

YOUR TURN · LIST COMPREHENSION

```
prices = [100, 140, 220]
with_tax = [round(p * 1.07) for p in prices]
print(with_tax)
```

Add 7% sales tax to every price. What prints?

A. `[107, 147, 227]`

B. `[107, 150, 235]`

C. 467

D. `[7, 10, 15]`

Day 21 3:00 PM · The back office · +100 rep · +$50 wages

How spread out? (analytics)



Theo

The owner says 'our average sale is $135, great!' But averages hide stuff. Show me how sale prices are spread out.

Big Face
That $520 sale is an outlier. Somebody overpaid.



THE BIG IDEA

An average hides a lot. A histogram counts how many sales fall in each price bucket. Make $50 buckets, then press Chart.

LIKE YOU'RE 10 Sorting coins into jars of $0–49, $50–99… and seeing which jar is fullest.

THE DETAIL

Look for skew and outliers before trusting a mean. Median (the middle value) is often more honest for prices.

EXAMPLE

```
SELECT MIN(price_usd), ROUND(AVG(price_usd)), MAX(price_usd) FROM sales;
```

WHAT IT RETURNS

MIN(price_usd)	ROUND(AVG(price_usd))	MAX(price_usd)
33	165	436

1 row

YOUR JOB

Histogram of sale prices: bucket = CAST(price_usd / 50 AS INTEGER) * 50, and sales = how many in each bucket. Lowest bucket first.

```
-- write it yourself
```

Hint: Group by the bucket you made.

SAME JOB IN PYTHON (PANDAS)

```
import matplotlib.pyplot as plt
sales['price_usd'].plot.hist(bins=range(0, 450, 50))
plt.xlabel('sale price (USD)'); plt.show()
sales['price_usd'].median()
```

PYTHON CORNER · LEARN IT FIRST · MIN, MAX, SPREAD

min() finds the smallest value in a list and max() finds the biggest.

max − min is called the range: how spread out the numbers are.

One weird value (an outlier) can stretch the range a lot, so always look at the data, not just the summary.

`resale = [210, 190, 205, 640]`	Four sales. 640 looks off.
`print(min(resale), max(resale))`	Smallest is 190, biggest is 640.
`print(max(resale) - min(resale))`	Range is 450, mostly because of that one 640.

OUTPUT

```
190 640
450
```

Remember: max - min = range. Outliers stretch it.

YOUR TURN · MIN, MAX, SPREAD

```
sales = [150, 180, 175, 520, 160]
print(max(sales) - min(sales))
```

What's the range (biggest minus smallest)?

A. 1185

B. 150

C. 370

D. 520

Day 22 4:45 PM · The back office · +100 rep · +$50 wages

Clean it, chart it (notebook)



Theo

Real data is messy. Download our notebook kit, open lesson 03 in Google Colab, clean the release list and draw a chart. Then tell me what you learned.

Big Face

Notebook done. My chart looks legit.



THE BIG IDEA

A notebook mixes notes, code and charts in one page. Google Colab runs Python notebooks free in your browser: no install.

LIKE YOU'RE 10 A notebook is a lab journal where the experiments actually run.

THE DETAIL

Open colab.research.google.com → Upload → choose a .ipynb from the kit, then drag the three .csv files into the Files panel on the left.

AFTER THE NOTEBOOK

Get the notebook kit from the Drop Table page, open `03_clean_and_chart.ipynb` in Google Colab (free), upload the three CSV files and run each cell.

1. Some retail prices are missing (NaN in pandas). What's the honest first step?

A. Replace them all with 0

B. Make up typical prices

C. Delete the whole table

D. Count them and decide on purpose: drop them or keep them as unknown

2. Why do this?

```
releases['release_date'] = pd.to_datetime(releases['release_date'])
```

A. To make the file smaller

B. So pandas treats the column as dates (weeks, months, sorting) instead of plain text

C. To delete the dates

D. It's required to print

3. A chart of drops per week should have...

A. weeks in time order on the x-axis and a title with the date range

B. random week order

C. only the biggest week

D. no labels

4. Your chart shows 34 drops in week 44. What should the caption say?

A. Which source, which dates, and when the list was checked

B. The exact sales for each drop

C. That every week is like this

D. Nothing, the chart speaks

PYTHON CORNER · LEARN IT FIRST · PANDAS

pandas is a Python tool for tables. A DataFrame is a table, like a SQL table, with named columns.

groupby("brand")["price"].mean() means: split rows into groups by brand, then average the price in each group. That's SQL's GROUP BY with AVG().

pandas needs the full Python install, so it runs in the Colab notebook kit, not this page. Below is the same idea done by hand with a dict, which does run here.

`rows = [("Kitebird", 100), ("Mavrik", 300), ("Kitebird", 200)]`	Three sales: (brand, price) pairs.
`groups = {}`	An empty dict to collect prices per brand.
`for brand, price in rows:`	Take each pair apart into brand and price.
`    groups[brand] = groups.get(brand, []) + [price]`	Add this price to that brand's list (start an empty list if it's new).
`for brand, prices in groups.items():`	Go through each brand and its prices.
`    print(brand, sum(prices) / len(prices))`	Average per brand. pandas does all of this in one line.

OUTPUT

```
Kitebird 150.0
Mavrik 300.0
```

Remember: groupby + mean = GROUP BY + AVG(). pandas runs in Colab.

YOUR TURN · PANDAS

```
import pandas as pd
df = pd.DataFrame({"brand": ["Kitebird", "Mavrik", "Kitebird"],
                   "price": [140, 300, 160]})
print(df.groupby("brand")["price"].mean())
```

What does Kitebird's row show?

A. `150.0`

B. `140`

C. `300.0`

D. 2



CHAPTER 5 · MACHINE LEARNING + AI

The AI lab

The AI lab. Big Face trains the same kind of models that pick what shows up in his feed, and learns why you always check an AI's work before trusting it.

WHY THIS MATTERS TO YOU

Your feed, face unlock, spam filters: that's machine learning. Today you train one yourself and see exactly how it decides.

Day 23: Line of best fit (SQL) · Day 24: Teach a model (gradient descent) · Day 25: Heat detector (decision tree) · Day 26: Customer groups (k-means) · Day 27: Your AI teammate · Day 28: Where data comes from · Day 29: Run the store (capstone) · Day 30: Grail Day (final review)

Day 23 10:00 AM · The AI lab · +150 rep · +$75 wages

Line of best fit (SQL)



Theo
Welcome to the AI lab. First question: does hype predict how much a shoe sells over retail? Draw the straightest line through the dots, using only SQL.

Big Face
A line that predicts prices? That's basically a cheat code.



THE BIG IDEA

A line of best fit is the simplest model. Its slope says how much the premium goes up for each extra point of hype.

LIKE YOU'RE 10 Drawing one straight line through a cloud of dots so you can guess where the next dot will land.

THE DETAIL

slope = covariance(x,y) / variance(x); intercept = mean(y) − slope × mean(x). Correlation isn't causation, and hype here is a made-up score.

EXAMPLE

```sql
WITH p AS (
  SELECT sh.hype AS x, 100.0 * (AVG(s.price_usd) - sh.retail_usd) / sh.retail_usd AS y
  FROM sales s JOIN shoes sh ON s.shoe_id = sh.shoe_id GROUP BY sh.shoe_id
)
SELECT x AS hype, ROUND(y) AS premium_pct FROM p ORDER BY x;
```

WHAT IT RETURNS

hype	premium_pct
1	-39
2	-40
2	-31
3	-44
3	-37

20 rows, first 5 shown

YOUR JOB

Fit premium_pct = intercept + slope × hype across the 20 shoes. Return one row: slope and intercept, each rounded to 1 decimal.

```
-- write it yourself
```

Hint: slope = SUM((x-mx)*(y-my)) / SUM((x-mx)*(x-mx))

SAME JOB IN PYTHON (PANDAS)

```
from sklearn.linear_model import LinearRegression
p = df.groupby(['shoe_id', 'hype', 'retail_usd'])['price_usd'].mean().reset_index()
p['premium_pct'] = 100 * (p['price_usd'] - p['retail_usd']) / p['retail_usd']
m = LinearRegression().fit(p[['hype']], p['premium_pct'])
m.coef_[0], m.intercept_
```

PYTHON CORNER · LEARN IT FIRST · A LINE IN PYTHON

A straight line has a rule: y = m × x + b. m is the slope (how much y goes up for each step in x) and b is where the line starts.

Machine learning's simplest model is exactly this. It looks at past data to choose m and b. Then you plug in a new x to get a guess for y.

Here x is hype (1 to 10) and y is how far over retail a shoe resells, in %.

`m = 5`	Slope: each hype point adds 5%.
`b = -10`	Start point: −10%.
`for hype in [2, 6, 10]:`	Try three hype scores.
`    guess = m * hype + b`	The line's rule.
`    print("hype", hype, "→ guess", guess)`	Shows each prediction.

OUTPUT

```
hype 2 → guess 0
hype 6 → guess 20
hype 10 → guess 40
```

Remember: Model = rule. Plug in x, get a guessed y.

YOUR TURN · A LINE IN PYTHON

```
m, b = 9.7, -68.6
hype = 8
premium = m * hype + b
print(round(premium))
```

The best-fit line from today. What premium % does it predict for hype 8?

A. 8

B. 9

C. 78

D. -69

Day 24 11:30 AM · The AI lab · +150 rep · +$75 wages

Teach a model (gradient descent)



Theo

Yesterday you used a formula. Real machine learning finds the line by trial and error: guess, check the mistake, nudge, repeat. Train it yourself and watch it learn.

Big Face

The model learns by stepping downhill. Too big a step and... boom.



THE BIG IDEA

Gradient descent starts with a bad line and improves it a little each step. The 'loss' is how wrong the line is. Learning means the loss goes down.

LIKE YOU'RE 10 Finding the bottom of a dark valley by feeling which way is downhill and taking small steps.

THE DETAIL

Each step moves the slope and intercept against the gradient of mean squared error. The learning rate sets the step size: too small is slow, too big overshoots and blows up.

THE LAB (RUNS LIVE ON THE PAGE)

On the page, a model starts with a flat line through the shoes (hype vs. how far over retail they sold) and improves it step by step. Try the three learning rates: slow, just right, and too big. Watch the loss curve.

CHECK-IN

What happened with the 'too big' learning rate?

A. Nothing changed

B. It learned fastest and best

C. The loss blew up: each step overshot the bottom

D. It found a different, better line

PYTHON CORNER · LEARN IT FIRST · ONE TRAINING STEP

How does a model choose its numbers? It starts with a guess, measures how wrong it is, and nudges the number to be less wrong. Repeat.

The gradient says which way is 'more wrong'. So you step the opposite way: new = old − learning_rate × gradient.

The learning rate is the step size. Too small: slow. Too big: it jumps past the answer.

`w = 1.0`	Current guess for the weight.
`lr = 0.5`	Step size.
`gradient = 2.0`	Positive gradient: going up makes it worse.
`w = w - lr * gradient`	1.0 − 0.5 × 2.0 = 0.0. We stepped down.
`print(w)`	Shows 0.0.

OUTPUT

```
0.0
```

Remember: new = old - lr * gradient. Step opposite the slope.

YOUR TURN · ONE TRAINING STEP

```
w = 0.0
lr = 0.1
gradient = -4.0
w = w - lr * gradient
print(w)
```

One step of gradient descent. What's the new weight?

A. 0.0
B. 4.0
C. -0.4
D. 0.4

Day 25 1:30 PM · The AI lab · +180 rep · +$90 wages

Heat detector (decision tree)



Theo
When a consigner walks in, can we predict on the spot if their pair will sell for 20%+ over retail? Train a decision tree on our sales log, and test it fairly.

Big Face
My heat detector beat the guess-everything baseline!



THE BIG IDEA

A decision tree asks yes/no questions (Is hype at least 7? Is the size 13+?) until it can guess HEAT or NOT. We train on 80% of sales and test on the other 20% it has never seen.

LIKE YOU'RE 10 Twenty Questions for shoes. Practice on some cards, then get quizzed on new cards.

THE DETAIL

This is classification. Test accuracy is the honest score. A very deep tree memorizes the training rows (overfitting): training accuracy goes up while test accuracy stalls or drops.

THE LAB (RUNS LIVE ON THE PAGE)

On the page, a decision tree learns to spot HEAT pairs (sold for 20%+ over retail) from 80% of the sales log, then gets tested on the 20% it never saw. Slide the depth from 1 to 10 and compare training vs. test accuracy.

CHECK-IN

Which tree depth should the store actually use?

A. Whichever trains fastest

B. The one with the best TEST accuracy

C. The deepest one, because training accuracy is highest

D. Depth 1, always

PYTHON CORNER · LEARN IT FIRST · ACCURACY

Accuracy is the share of guesses that were right: right ÷ total.

zip(a, b) pairs up two lists item by item: first with first, second with second.

In Python, True counts as 1 and False as 0, so sum() of a bunch of True/False answers counts the Trues.

`guess = ["heat", "brick", "heat"]`	The model's three guesses.
`real  = ["heat", "heat", "heat"]`	What really happened.
`for g, r in zip(guess, real):`	Walk both lists side by side.
`    print(g, r, g == r)`	Show each pair and whether it matched.
`print(sum(g == r for g, r in zip(guess, real)))`	Count the matches: 2 of 3.

OUTPUT

```
heat heat True
brick heat False
heat heat True
2
```

Remember: zip pairs lists up. sum of True/False counts the Trues.

YOUR TURN · ACCURACY

```
predicted = [1, 0, 1, 1, 0]
actual    = [1, 0, 0, 1, 1]
right = sum(p == a for p, a in zip(predicted, actual))
print(right / len(actual))
```

How accurate was the heat detector?

A. `0.8`

B. `3`

C. `0.6`

D. `0.4`

Day 26 3:15 PM · The AI lab · +150 rep · +$75 wages

Customer groups (k-means)



Theo
Marketing wants to send different emails to different kinds of buyers. Nobody labeled them. Can a model find natural groups on its own?

Big Face
Groups nobody labeled. The computer found them itself.



THE BIG IDEA

k-means finds groups without labels. You pick how many groups (k). It drops k centers, assigns every sale to the nearest center, moves each center to the middle of its group, and repeats.

LIKE YOU'RE 10 Sorting a pile of mixed Legos into groups that look alike, without anyone telling you the groups.

THE DETAIL

This is unsupervised learning. We cluster on shoe size and price ÷ retail (scaled so both count equally). Different k gives different stories; there's no single 'right' k.

THE LAB (RUNS LIVE ON THE PAGE)

On the page, k-means sorts every sale into groups by shoe size and price ÷ retail, with no labels at all. Change k and press 'Run to the end' to watch the centers move.

CHECK-IN

What does k-means need from you?

A. Nothing at all
B. How many groups (k) to look for
C. The right answer for every sale
D. A list of customer names

PYTHON CORNER · LEARN IT FIRST · DISTANCE

To group similar sales, a computer needs to measure how far apart two points are.

Distance is the Pythagorean rule: square the gap in each direction, add them up, then take the square root. In Python, ** is 'to the power of', and ** 0.5 is a square root.

A tuple like (3, 4) is a list that can't change. a[0] is its first number, a[1] its second.

`a = (0, 0)`	Point a.
`b = (3, 4)`	Point b.
`gap_x = b[0] - a[0]`	Gap across: 3.
`gap_y = b[1] - a[1]`	Gap up: 4.
`print(gap_x ** 2, gap_y ** 2)`	Squared: 9 and 16.
`print((gap_x ** 2 + gap_y ** 2) ** 0.5)`	√(9 + 16) = √25 = 5.0.

OUTPUT

```
9 16
5.0
```

Remember: distance = (dx**2 + dy**2) ** 0.5

YOUR TURN · DISTANCE

```
a = (10, 1.2)   # size, price ÷ retail
b = (13, 1.6)
d = ((a[0]-b[0])**2 + (a[1]-b[1])**2) ** 0.5
print(round(d, 2))
```

k-means measures how far apart two sales are. What prints?

A. `3.4`

B. `9.16`

C. `0.4`

D. `3.03`

Day 27 4:30 PM · The AI lab · +120 rep · +$60 wages

Your AI teammate



Theo

We use an AI assistant to draft SQL. It's fast, and sometimes confidently wrong. It wrote this query for 'all Jordan releases in December 2026' and got zero rows. Find the bug and fix it.

Big Face
The AI was confident... and wrong. Always check the count.



THE BIG IDEA

AI chat tools (large language models) write text and code by predicting likely words. Machine learning models like your tree learn from our own data. Both can be wrong, so you check the output.

LIKE YOU'RE 10 An AI helper is like a fast new coworker who never says 'I'm not sure'. Double-check their work.

THE DETAIL

Verify AI-written queries: run them, check row counts, check values against the data. Never paste secrets into an AI. The store's models only know the period they were trained on.

THE AI'S DRAFT

```
-- AI draft (has a bug):
SELECT release_date, name, retail_usd
FROM releases
WHERE brand = 'Air Jordan'
  AND release_date BETWEEN '2026-12-01' AND '2026-12-31';
```

WHAT IT RETURNS

(no rows)

YOUR JOB

Fix the AI's query so it returns every Jordan release in December 2026: release_date, name, retail_usd.

```
-- write it yourself
```

Hint: Run SELECT DISTINCT brand FROM releases; to see how brands are spelled.

SAME JOB IN PYTHON (PANDAS)

```
releases['brand'].unique()  # always look at the real values before you filter
```

PYTHON CORNER · LEARN IT FIRST · CHECK THE AI

AI tools can sound confident and still be wrong about numbers. The fix is easy: check against the data yourself.

.count(x) tells you how many times x appears in a list.

Then compare the AI's claim to the real count with ==. True means the AI was right.

`sales = ["Glide", "Dash", "Glide", "Glide"]`	The real data.
`ai_claim = 2`	What the AI told us.
`real = sales.count("Glide")`	Count it ourselves: 3.
`print(real, real == ai_claim)`	3 vs the claim of 2: False. The AI was wrong.

OUTPUT

```
3 False
```

Remember: Trust, then verify: count it yourself and compare.

YOUR TURN · CHECK THE AI

```
rows = ["Jordan", "Nike", "Jordan", "adidas"]
ai_says = 3
real = rows.count("Jordan")
print(real == ai_says)
```

The AI said there were 3 Jordan rows. Was it right?

A. `False`

B. 3

C. `True`

D. 2

Day 28 5:30 PM · The AI lab · +80 rep · +$40 wages

Where data comes from



Theo

Last lesson before you run a shift alone: where our numbers come from, and the rules for collecting them.

Big Face
Real data has a source and a date. Got it.



THE BIG IDEA

The release calendar here is real. A scheduled helper checks a public release calendar every day and writes down what it finds, with the date it checked. Websites have rules about robots, and good data people follow them.

LIKE YOU'RE 10 Copying the lunch menu off the school board every morning, and writing the date at the top.

THE DETAIL

Official APIs (documented, keyed, rate-limited) beat scraping (fragile, and often against a site's terms). Marketplaces like StockX and GOAT don't allow automated scraping. Always record source, collection time and coverage window.

QUICK CHECK

1. Why does every dataset here show a date range and a 'checked on' time?

A. So you know what period the numbers cover and how fresh they are
B. To make the table longer
C. Computers need it to run
D. It looks cool

2. A site's terms say 'no automated scraping'. What do you do?

A. Scrape it faster
B. Use an official API or a source that allows it
C. Scrape at night
D. Pretend to be a person

3. Can you copy product photos from a sneaker news site onto your own page?

A. Not without permission: they're copyrighted. Link to the article instead
B. Yes, everything online is free
C. Yes, if you resize them
D. Yes, if it's for school

4. An API key is like...

A. the price of a shoe
B. a type of SQL
C. a username anyone can see
D. a password: keep it secret, never in public code

5. The sales log in week 3 is...

A. practice data made up for learning
B. live auction data
C. real store prices
D. real StockX prices

PYTHON CORNER · LEARN IT FIRST · DATES

Dates aren't just text. Python's datetime tool lets you do math with them.

from datetime import date brings in the date tool. date(2026, 12, 1) means year, month, day, in that order.

Subtracting two dates gives a gap. .days turns that gap into a plain number of days.

`from datetime import date`	Bring in the date tool.
`start = date(2026, 11, 1)`	Nov 1, 2026.
`end = date(2026, 11, 15)`	Nov 15, 2026.
`gap = end - start`	Later minus earlier.
`print(gap.days)`	14 days.

OUTPUT

```
14
```

Remember: date(y, m, d). later - earlier gives a gap; .days is the number.

YOUR TURN · DATES

```
from datetime import date
drop = date(2026, 11, 27)
today = date(2026, 11, 5)
print((drop - today).days)
```

How many days until the drop?

A. 22

B. 5

C. -22

D. 27

Day 29 6:00 PM · The AI lab · +250 rep · +$125 wages

Run the store (capstone)



Theo

You've earned the keys. Pick a real question the store has, answer it with data, and give me a one-sentence finding with the time period. Then you're Store Legend.

Big Face
Run the whole store? Let's get it. Grail's waiting.



THE BIG IDEA

Ask your own question, write the SQL, chart it if it helps, and explain the answer in one or two sentences. Say which time period you used and which data (real or practice).

LIKE YOU'RE 10 Your science fair project: a question, a test, an answer, and what you'd check next.

YOUR JOB: RUN THE STORE

Ask your own question, write the SQL, and explain the answer in one or two sentences. Always say which time period you used and which data (real or practice).

```
-- Ideas:
-- Which week in December has the most Jordan releases?
-- Do bigger sizes resell for more? (sales.size_us)
-- Which brand has the most HEAT shoes?
SELECT size_us, ROUND(AVG(price_usd)) AS avg_price, COUNT(*) AS sales
FROM sales GROUP BY size_us ORDER BY size_us;
```

WHAT THE STARTER QUERY RETURNS

size_us	avg_price	sales
4	195	21
5	187	33
6	176	52
7	158	90
7.5	186	82
8	153	154

16 rows, first 6 shown

PYTHON CORNER · LEARN IT FIRST · PUT IT TOGETHER

A while loop repeats as long as its test is True. It's for when you don't know how many times you'll need.

Each time through, something inside should change, or the loop never ends.

A counter (weeks += 1) keeps track of how many times it ran.

`jar = 10`	Starting cash in the jar.
`days = 0`	Counter starts at 0.
`while jar < 40:`	Keep going while the jar is under 40.
`    jar += 12`	Save 12 each day.
`    days += 1`	Count the day.
`    print("day", days, "jar", jar)`	Show progress.
`print("Done in", days, "days")`	Runs once the test turns False.

OUTPUT

```
day 1 jar 22
day 2 jar 34
day 3 jar 46
Done in 3 days
```

Remember: while test: repeats until the test is False. Change something inside!

YOUR TURN · PUT IT TOGETHER

```
wallet = 380
grail = 450
weekly = 45
weeks = 0
while wallet < grail:
    wallet += weekly
    weeks += 1
print(weeks)
```

How many weeks until Big Face can buy his grail?

A. `70`

B. 1

C. 3

D. 2

Day 30 7:00 PM · The AI lab · +200 rep · +$100 wages

Grail Day (final review)



Theo
Big Face counts the jar twice. Big face hundreds, finally enough for the grail. Rae stops him at the register: "One last thing. Mixed questions from every week, no notes. Then the Comet Mid is yours."

Big Face
Five weeks, one grail. Big face hundreds, all mine. Let's cop it.



THE BIG IDEA

A final review mixes questions from all five weeks: SQL, Python, data, machine learning and AI. Pulling an idea back out of memory, after time has passed and mixed in with other ideas, is one of the best-studied ways to make it stick.

LIKE YOU'RE 10 Like the end-of-season game: every play you practiced, all in one night.

THE DETAIL

If one question stumps you, that's useful information, not failure. Go back to that day, reread the big idea, then come back. Each question links to a week below in its explanation.

QUICK CHECK

1. Week 1–2 · Which query finds releases that have no price yet?

A. SELECT name FROM releases WHERE retail_usd IS NULL;

B. SELECT name FROM releases WHERE retail_usd = '';

C. SELECT name FROM releases WHERE retail_usd = 0;

D. SELECT name FROM releases WHERE retail_usd = NULL;

2. Week 2 · You want only brands with at least 5 releases. Which part filters the groups?

A. HAVING COUNT(*) >= 5

B. LIMIT 5

C. WHERE COUNT(*) >= 5

D. ORDER BY COUNT(*) >= 5

3. Week 3 · sales has shoe_id. shoes has shoe_id and model. To show each sale next to its model you…

A. JOIN shoes ON sales.shoe_id = shoes.shoe_id

B. Copy the model into every sale by hand

C. SELECT both tables and hope

D. GROUP BY shoe_id

4. Week 4 · What prints?

```
closet = ["Glide 1", "Dash 5"]
closet.append("Comet Mid")
print(closet[-1])
```

A. 3

B. Dash 5

C. Glide 1

D. Comet Mid

5. Week 4 · What prints?

```
prices = [120, 300, 90]
print([p for p in prices if p > 100])
```

A. [120, 300, 90]

B. [120, 300]

C. [90]

D. 2

6. Week 4 · What prints?

```
pair = {"model": "Comet Mid", "paid": 410}
print(pair.get("hype", 0))
```

A. 410

B. 0

C. An error

D. None

7. Week 4 · Most sales are about $165, but one was $5,000. Which number best shows a typical sale?

A. The median

B. The max

C. The mean (average)

D. The range

8. Week 5 · The heat detector scores 90% accuracy, but 90% of shoes are bricks. That means…

A. It's an amazing model

B. It needs more hype

C. Accuracy can't be 90%

D. It might be no better than always guessing 'brick'

9. Week 5 · The AI says there are 12 Jordan releases in December. Best next step?

A. Post it on the blog

B. Trust it, AI is always right

C. Ask the AI if it's sure

D. Run a COUNT(*) query yourself and compare

PYTHON CORNER · LEARN IT FIRST · GRAIL DAY: DICTS + FUNCTIONS

Real programs mix everything you've learned. Today's puzzle uses a dict, a function, a loop and math together. Read it in pieces.

.items() loops over a dict and hands you each key and value together, so for name, cash in jar.items(): gives you both at once.

A function can loop and add things up inside, then return the total. Code outside the function only sees what comes back.

`jar = {"Mon": 12, "Tue": 0, "Wed": 20}`	A dict: day → cash saved.
`def total_saved(days):`	A recipe that takes a dict of days.
`    total = 0`	Start a running total inside the function.
`    for day, cash in days.items():`	.items() gives each day and its cash together.
`        if cash > 0:`	Only mention days where he saved something.
`            print(day, "saved", cash)`	Prints Mon and Wed (Tue was 0).
`        total += cash`	Add every day's cash, even 0.
`    return total`	Hand the total back.
`print("Total:", total_saved(jar))`	Call the function and print what it returns: 32.

OUTPUT

```
Mon saved 12
Wed saved 20
Total: 32
```

Remember: Break big code into pieces: what's the data, what does the function do, what comes back?

YOUR TURN · GRAIL DAY: DICTS + FUNCTIONS

```
closet = {"Glide 1": 100, "Dash 5": 150, "Orbit 3": 130}

def sell_all(pairs, fee=0.10):
    total = 0
    for model, price in pairs.items():
        total += price * (1 - fee)
    return round(total)

wallet = 210
wallet += sell_all(closet)
print(wallet >= 540, wallet)
```

Big Face sells his whole closet (10% app fee) to buy the $540 grail. What prints?

A. `False 552`

B. `True 552`

C. `False 210`

D. `True 590`

Appendix A · SQL cheat sheet

SELECT * FROM

Day 2

Show every column of a table.

```
SELECT * FROM releases;
```

SELECT col, col

Day 3

Show only the columns you name.

```
SELECT name, retail_usd FROM releases;
```

ORDER BY … DESC

Day 4

Sort. ASC = small to big (default), DESC = big to small.

```
SELECT name, retail_usd FROM releases ORDER BY
retail_usd DESC;
```

LIMIT

Day 4

Keep only the first N rows.

```
SELECT name FROM releases LIMIT 5;
```

WHERE

Day 5

Keep rows that match a rule. Text goes in 'single quotes'.

```
SELECT name FROM releases WHERE brand =
'Jordan';
```

AND / OR

Day 5

Combine rules. AND = both true, OR = either.

```
SELECT name FROM releases WHERE brand = 'Nike'
AND retail_usd < 150;
```

LIKE '%word%'

Day 6

Search inside text. % means 'anything here'.

```
SELECT name FROM releases WHERE name LIKE
'%Dunk%';
```

BETWEEN … AND

Day 7

A range, including both ends. Great for dates.

```
SELECT name FROM releases WHERE release_date
BETWEEN '2026-11-01' AND '2026-11-30';
```

date('now')

Day 7

Today's date (UTC). Add days with date('now', '+7 days').

```
SELECT name FROM releases WHERE release_date
<= date('now', '+7 days');
```

IS NULL

Day 8

Find blanks. Never write = NULL.

```
SELECT name FROM releases WHERE retail_usd IS
NULL;
```

COUNT / AVG / MIN / MAX / SUM

Day 9

Squash many rows into one number.

```
SELECT COUNT(*), AVG(retail_usd) FROM
releases;
```

AS

Day 9

Give a result column a nickname.

```
SELECT AVG(retail_usd) AS avg_price FROM
releases;
```

ROUND(x, 2)

Day 9

Round a number.

```
SELECT ROUND(AVG(retail_usd), 2) FROM
releases;
```

GROUP BY

Day 10

One answer per group.

```
SELECT brand, COUNT(*) FROM releases GROUP BY
brand;
```

strftime('%W', date)

Day 11

Pull a piece out of a date: %Y year, %m month, %W week.

```
SELECT strftime('%W', release_date) AS wk,
COUNT(*) FROM releases GROUP BY wk;
```

HAVING

Day 12

Filter groups (WHERE filters rows, HAVING filters groups).

```
SELECT release_date, COUNT(*) FROM releases
GROUP BY release_date HAVING COUNT(*) >= 4;
```

JOIN … ON

Day 14

Glue two tables where an ID matches.

```
SELECT sh.model, s.price_usd FROM sales s JOIN shoes sh ON s.shoe_id = sh.shoe_id;
```

CASE WHEN … THEN … ELSE … END

Day 15

Make a label from a rule.

```
SELECT price_usd, CASE WHEN price_usd > 300 THEN 'heat' ELSE 'normal' END FROM sales;
```

WITH name AS (…)

Day 16

Save a query as a temporary table, then use it.

```
WITH avgp AS (SELECT AVG(price_usd) AS a FROM sales) SELECT COUNT(*) FROM sales, avgp WHERE price_usd > a;
```

RANK() OVER (ORDER BY …)

Day 17

Number rows 1, 2, 3 by a value (a window function).

```
SELECT brand, COUNT(*) AS n, RANK() OVER (ORDER BY COUNT(*) DESC) FROM shoes GROUP BY brand;
```

Appendix B · Every column

releases

Column	Type	What it means	Example
id	number	Row number we gave each release.	12
release_date	text 'YYYY-MM-DD'	The day it drops. Always in quotes, year first, so sorting works.	'2026-11-14'
brand	text	Who makes it: Nike, Jordan, adidas, New Balance… Spelled like the list, so 'Jordan' not 'Air Jordan'.	'Jordan'
name	text	Full release name. The nickname is in double quotes inside it.	Air Jordan 11 "Green Screen"
colorway	text or NULL	The official colors, split by slashes. NULL when not announced yet.	'White/Black-Infrared'
style_code	text or NULL	The brand's product code, like a SKU. Same code = same shoe.	'CT8529-105'
retail_usd	number or NULL	Store price in US dollars. NULL means the price isn't announced (use IS NULL to find them).	225

shoes

Column	Type	What it means	Example
shoe_id	number	ID for each practice shoe. Links to sales.shoe_id.	4
brand	text	Made-up brand: Northbound, Halo Athletic, Kitebird or Mavrik.	'Kitebird'
model	text	Model name.	'Glide 1'
colorway	text	Colors.	'Lava'
release_date	text 'YYYY-MM-DD'	When this practice shoe came out.	'2026-07-18'
retail_usd	number	Store price in dollars.	160
pairs_made	number	How many pairs exist. Fewer pairs = rarer.	8000
hype	number 1–10	How excited people are. 10 = everyone wants it.	9

sales

Column	Type	What it means	Example
sale_id	number	ID for each sale.	301
shoe_id	number	Which shoe sold. Match it to shoes.shoe_id with a JOIN.	4
sale_date	text 'YYYY-MM-DD'	Day it sold (Aug 1 – Oct 31, 2026).	'2026-09-12'
size_us	number	US men's size.	10.5
price_usd	number	What the buyer paid, in dollars.	245
condition	text	'new' (never worn, called deadstock) or 'used'.	'new'

Appendix C · Money math

Every curveball in the game is a little math problem. Here are 15 of the 100.

BAD BREAK

Cracked phone screen

Big Face's phone slips out of his pocket at the bus stop. The repair shop charges $40.

`Wallet -$40.`

BAD BREAK

Hole in the pocket

A hole in his jacket pocket. Some of the cash he was carrying is gone.

`15% of the wallet. On $200 that's $30.`

BAD BREAK

Crushed box

A pair's box got crushed in the closet. Collectors pay more for a clean box, so this one is worth less.

`One pair's value × 0.85: a $180 pair becomes $153.`

BAD BREAK

Kitebird restock

Kitebird announces a big restock. More pairs means lower prices for everyone holding them.

`Kitebird prices × 0.8: a $200 pair becomes $160.`

BAD BREAK

Chargeback scam

A buyer said a pair 'never arrived' even though tracking said delivered. The app sided with the buyer this time.

`Wallet -$45.`

BAD BREAK

Sales tax surprise

A $100 cleaning kit order came to $107. Sales tax is 7% here: $7 more than the sticker price.

`Wallet -$7.`

BAD BREAK

Instant transfer fees

Big Face kept paying 'instant transfer' fees instead of waiting a day. They added up to about 5%.

`5% of the wallet. On $200 that's $10.`

LUCKY BREAK

Holiday bonus

Rae gives every worker a $30 holiday bonus.

`Wallet +$30.`

LUCKY BREAK

Savings match

Mom matches 10% of whatever Big Face has saved.

`10% of the wallet. On $200 that's $20.`

LUCKY BREAK

Interest paid

His savings account pays 2% interest. Small, but it's free money for doing nothing.

`2% of the wallet. On $200 that's $4.`

YOUR CALL

Double your money?

A stranger DMs: 'Send $20, I'll send back $40 in an hour. Guaranteed.'

Send $20 or Block and report

`A choice: think about the chance it works and what you lose if it doesn't.`

YOUR CALL

Ship with insurance?

Big Face's mailing a sold pair. Insurance costs $4. Without it, if the package is lost, he has to refund the buyer $60.

Pay $4 for insurance or Skip it

`A choice: think about the chance it works and what you lose if it doesn't.`

YOUR CALL

Pro cleaning

A pro cleaner offers to restore one of your pairs for $15. Clean pairs sell for about 20% more.

Pay $15 or Clean it yourself

`A choice: think about the chance it works and what you lose if it doesn't.`

YOUR CALL

Quick cash offer

A buyer offers to take one of your pairs right now at 90% of market price. No fees, no waiting.

Take it or Hold

`A choice: think about the chance it works and what you lose if it doesn't.`

YOUR CALL

Bulk cleaner deal

Cleaner is $13 a bottle, or 3 bottles for $30.

Buy 3 for $30 or Buy 1 for $13

`A choice: think about the chance it works and what you lose if it doesn't.`

The three formulas Big Face uses most

```
you_get = sale_price * (1 - 0.10)        # after the 10% app fee
profit  = you_get - what_you_paid
percent = (new - old) / old * 100          # how much a price moved
```

Appendix D · Glossary

118 words from the course. The number after each word is the day it's taught.

+= and -= `Python · Day 4`
Shortcuts for changing a variable: x += 5 means x = x + 5.

== (equals?) `Python · Day 5`
Asks whether two values are equal. One = stores; two == asks.

*** (star)** `SQL · Day 2`
In SELECT, * means every column.

Aggregate function `SQL · Day 9`
A function that squashes many rows into one number: COUNT, SUM, AVG, MIN, MAX.

AND / OR `SQL · Day 5`
Combine tests. AND needs both to be true; OR needs at least one.

API key `Data basics · Day 28`
A secret code that identifies you to an API. Treat it like a password: never put it in public code.

AS (alias) `SQL · Day 9`
Gives a result column a nickname so the answer is easy to read.

Average (mean) `Analytics · Day 9`
Add the values, divide by how many. One extreme value can pull it a long way.

Baseline `Machine learning · Day 25`
The score you'd get by always guessing the most common answer. A model has to beat it to be useful.

Boolean `Python · Day 5`
A True or False value. Comparisons give booleans. In SQL and Python they answer yes/no questions.

Classification `Machine learning · Day 25`
A model that sorts things into groups you named in advance, like HEAT or NOT.

Column `Data basics · Day 2`
One kind of fact that every row has, like brand or retail_usd. Also called a field.

CSV `Data basics · Day 22`
Comma-separated values: a plain-text table where each line is a row and commas split the columns.

// and % `Python · Day 12`
// divides and keeps only the whole number. % gives the remainder.

.items() `Python · Day 30`
Loops over a dict and gives each key and value together.

Accuracy `Machine learning · Day 25`
The share of guesses that were right: right ÷ total.

AI (artificial intelligence) `AI · Day 27`
Computer systems that do things that seem smart, like answering questions or spotting patterns.

API `Data basics · Day 28`
A documented way for programs to ask another service for data, with rules about how often you can ask.

append() `Python · Day 18`
Adds an item to the end of a list.

Assignment (=) `Python · Day 2`
Storing a value in a variable. The right side runs first, then the answer goes into the name on the left.

AVG `SQL · Day 9`
The average: add the values up, divide by how many. NULLs are skipped.

BETWEEN `SQL · Day 7`
Keeps values in a range, including both ends. Great for dates.

CASE WHEN `SQL · Day 15`
SQL's if/else: gives a value depending on a test.

Cluster `Machine learning · Day 26`
A group of similar points that k-means finds.

COUNT `SQL · Day 9`
Counts rows. COUNT(*) counts every row; COUNT(column) skips NULLs.

Data `Data basics · Day 1`
Facts written down so you can look at them later: a price, a date, a size.

Data cleaning Analytics · Day 22
Fixing messy data before analysis: spelling, missing values, duplicates, wrong types.

Database Data basics · Day 2
A collection of tables that a program can search quickly. The Grail Room's database holds releases, shoes and sales.

Date format (YYYY-MM-DD) SQL · Day 7
Writing dates year-month-day so they sort in the right order, like words in a dictionary.

Decision tree Machine learning · Day 25
A model that asks yes/no questions until it can make a guess.

DESC SQL · Day 4
Short for descending: sort biggest first.

f-string Python · Day 10
Text with an f in front, where {curly brackets} get replaced by values.

Float Python · Day 9
A number with a decimal point, like 180.0. Dividing with / always gives a float.

FROM SQL · Day 2
Says which table to look in.

Gradient descent Machine learning · Day 24
Training by small steps: measure the loss, step downhill, repeat.

Hallucination AI · Day 27
When an AI states something false with confidence. Always check its numbers against the data.

Histogram Analytics · Day 21
A bar chart that counts how many values land in each bucket, like $50 price ranges.

import Python · Day 22
Brings a library into your program.

Indent Python · Day 7
Spaces at the start of a line. In Python they show which lines belong inside an if, for, def or while.

Data type Data basics · Day 1
What kind of fact a value is: text, number, date, true/false, or missing. You need to know the type before you can compare or add.

DataFrame Python · Day 20
A table in pandas, with named columns.

datetime Python · Day 28
Python's library for dates. Subtracting two dates gives a gap in days.

def Python · Day 17
Starts a function you write yourself.

Dictionary (dict) Python · Day 19
Values stored under labels called keys, like one row of a table.

Feature Machine learning · Day 25
An input a model uses to guess, like hype or size.

for loop Python · Day 15
Repeats some lines once for each item in a list.

Function Python · Day 1
A named command that does a job. You call it with brackets: print(…), len(…). You can write your own with def.

GROUP BY SQL · Day 10
Makes piles: one pile per value (one per brand, say), then an aggregate runs once per pile.

HAVING SQL · Day 12
Filters groups after GROUP BY. WHERE filters rows before grouping.

if / elif / else Python · Day 7
Makes a decision. The indented lines under if run only when its test is True; else runs otherwise.

in Python · Day 6
Checks whether something is inside something else: text inside text, an item in a list, a key in a dict.

Index Python · Day 13
An item's position number in a list. Python starts at 0; -1 means the last item.

Integer Python · Day 9
A whole number, like 60. Also called int.

JOIN SQL · Day 14
Glues rows from two tables together when their keys match.

Key SQL · Day 14
A column that identifies a row, like shoe_id. Tables that share a key can be joined.

Large language model (LLM) AI · Day 27
An AI that writes text and code by predicting likely next words. Chat assistants are LLMs.

len() and sum() Python · Day 14
len() counts the items in a list; sum() adds them up. sum/len is the average.

LIKE SQL · Day 6
Searches inside text. % stands for 'anything here'.

Line of best fit Machine learning · Day 23
The straight line that sits closest to all the points. y = m × x + b.

List comprehension Python · Day 20
Builds a new list in one line.

Machine learning Machine learning · Day 23
Getting a computer to learn a model from examples instead of writing the rule by hand.

Method Python · Day 6
A function attached to a value with a dot, like name.lower() or closet.append(x).

None Python · Day 8
Python's word for missing, like SQL's NULL. Test it with is None.

NULL SQL · Day 8
SQL's word for missing or unknown. It isn't zero and it isn't empty text.

Operator Python · Day 3
A symbol that does math or comparisons: + - * / // % ** and > < == !=.

Outlier Analytics · Day 21
A value way off from the rest, like one $520 sale among $150s.

IS NULL SQL · Day 8
The way to test for missing values. = NULL never matches anything.

k-means Machine learning · Day 26
Finds k groups in data without labels by moving group centers until they settle.

Label Machine learning · Day 25
The answer you want a model to learn, like HEAT or NOT.

Learning rate Machine learning · Day 24
The size of each gradient descent step. Too big and the model overshoots.

Library Python · Day 20
Code someone else wrote that you bring in with import, like pandas or datetime.

LIMIT SQL · Day 4
Keeps only the first N rows of the answer.

List Python · Day 13
Many values in order, inside square brackets.

Loss Machine learning · Day 24
A number for how wrong a model is right now. Training tries to make it smaller.

Median Analytics · Day 21
The middle value when everything is sorted. One outlier barely moves it.

Model Machine learning · Day 23
A rule learned from data that turns an input into a guess, like 'hype 8 → about 9% premium'.

Notebook Python · Day 22
A page that mixes notes, code and charts. Google Colab runs them free in the browser.

ON SQL · Day 14
Says which columns must match in a JOIN.

ORDER BY SQL · Day 4
Sorts the answer. ASC goes small to big (the default); DESC goes big to small.

Overfitting Machine learning · Day 25
When a model memorizes its training data and does worse on new data.

pandas Python · Day 20
A Python library for tables. Same ideas as SQL, written as Python.

Practice data Data basics · Day 13
Made-up data built for learning, so you can practice without real people's information. Here: the sales log and brands in weeks 3–5.

Premium Analytics · Day 15
How much more (or less) a shoe resells for than its store price. Positive = HEAT.

Prompt AI · Day 27
What you type to an AI. Clear prompts with context get better answers.

Query SQL · Day 2
One question written in SQL. It ends with a semicolon.

RANK() SQL · Day 17
Hands out 1st, 2nd, 3rd place. Ties share a place.

return Python · Day 17
Hands an answer back from a function to whoever called it.

round() Python · Day 11
Rounds a number. round(x, 2) keeps two decimal places.

Scraping Data basics · Day 28
Having a program copy data off web pages. Many sites forbid it in their terms, so use an API or a source that allows it.

Slope Machine learning · Day 23
How steep a line is: how much y goes up for each step in x.

SQL SQL · Day 2
Structured Query Language: the language for asking a database questions. You write a query; the database answers with a table.

strftime SQL · Day 11
A SQLite function that pulls pieces out of a date: %Y year, %m month, %W week number.

Table Data basics · Day 2
Data arranged in rows and columns, like a wall of shoe boxes. Each table has a name, like releases.

Parameter Python · Day 17
A blank in a function that you fill in when you call it. It can have a default value.

Prediction Machine learning · Day 23
A model's guess for a new input.

print() Python · Day 1
Shows a value on the screen.

Python Python · Day 1
A popular programming language that reads almost like English. It runs one line at a time, top to bottom.

Range Analytics · Day 21
Biggest minus smallest: how spread out the values are.

Real data Data basics · Day 28
Data collected from the real world. Here: the release calendar, with the date it was checked.

ROUND SQL · Day 9
Rounds a number to a number of decimal places.

Row Data basics · Day 1
One thing in a table: one shoe, one sale. Also called a record.

SELECT SQL · Day 2
Means 'show me'. It lists the columns you want back.

Source Data basics · Day 28
Where data came from. Good data always says its source, when it was collected and what period it covers.

Standardize Machine learning · Day 26
Rescale features so one with big numbers (like price) doesn't drown out one with small numbers (like size).

String Python · Day 1
Text inside quotes. The quotes mark where it starts and ends; they don't print.

Test data Machine learning · Day 25
Examples held back from training so you can check the model on data it has never seen. Test accuracy is the honest score.

Training data Machine learning · Day 24
The examples a model learns from.

Tuple Python · Day 26
Like a list in round brackets that can't be changed.

Variable Python · Day 2
A labeled box that holds a value. = puts a value in the box.

WHERE SQL · Day 5
Keeps only the rows that pass a test. Text values go in single quotes.

Wildcard SQL · Day 6
A symbol that matches anything. In LIKE, % matches any number of characters.

WITH (CTE) SQL · Day 16
Builds a mini-table first, names it, then uses it in the main query. CTE = common table expression.

Trend Analytics · Day 11
How a number changes over time, like drops per week.

Unsupervised learning Machine learning · Day 26
Learning patterns from data that has no labels, like k-means finding groups on its own.

Verify AI · Day 27
Check a claim yourself: run the query, count the rows, compare.

while loop Python · Day 29
Repeats lines as long as a test stays True. Something inside has to change, or it never stops.

Window function SQL · Day 17
A function that looks across many rows without squashing them into one, like RANK() OVER (...).

zip() Python · Day 25
Pairs up two lists item by item.

Answer key

Results use the release list from November 5, 2026. On the live page your row counts can differ as the list updates; what matters is that your query asks the right question.

DAY 1 · WHAT IS DATA?

1. D: Retail price: $130. A price is a fact you can write down and compare.

2. B: a date. Year-month-day is a date. Written this way it also sorts in order.

3. A: missing (NULL). Missing isn't zero. A $0 shoe and a shoe with no price yet are different things.

4. B: row. Each row is one thing (one shoe). Each column is one fact about it (its price).

DAY 1 · PYTHON CORNER

D: `Big Face clocks in`. print() shows whatever is inside the brackets. The quotes just mark where the text starts and ends.

DAY 2 · TABLES ARE SHOE BOXES

```
SELECT * FROM releases LIMIT 10;
```

id	release_date	brand	name	colorway	style_code	retail_usd
1	2026-11-05	Nike	Victor Wembanyama x Nike GT Cut 4 "Statue of Liberty"	Mint Foam/Metallic Gold	IW0737-300	210
2	2026-11-05	Nike	Nike Air Force 1 "What The LA"	White/Black-Hyper Jade	CT1117-100	125
3	2026-11-05	Nike	Nike Air Max 95 GORE-TEX "Black"	Black/Diffused Blue	IO1877-002	220
4	2026-11-05	Nike	Nike Air Force 1 Mid GTX Vibram "Black"	Black/Smoke Grey	IO1875-001	180

10 rows, first 4 shown

DAY 2 · PYTHON CORNER

A: `60`. A variable is a labeled box. wallet holds 60, so print(wallet) shows 60.

DAY 3 · PICK YOUR COLUMNS

```
SELECT name, retail_usd FROM releases;
```

name	retail_usd
Victor Wembanyama x Nike GT Cut 4 "Statue of Liberty"	210
Nike Air Force 1 "What The LA"	125
Nike Air Max 95 GORE-TEX "Black"	220
Nike Air Force 1 Mid GTX Vibram "Black"	180

111 rows, first 4 shown

DAY 3 · PYTHON CORNER

C: 75. 60 + 15 = 75. The new value goes back into the wallet box.

DAY 4 · SORT BY PRICE

```
SELECT name, retail_usd FROM releases ORDER BY retail_usd DESC LIMIT 5;
```

name	retail_usd
Air Jordan 4 "BIN 23"	355
Victor Solomon x Nike Air Foamposite One "Trophies Red"	350
Pharrell Williams x adidas Adistar Jellyfish "Dark Brown Frozen Green"	300
Pharrell x adidas Adistar Jellyfish "Auburn Timber"	300

5 rows, first 4 shown

DAY 4 · PYTHON CORNER

A: 83. += adds to what's there, -= subtracts. 75 + 20 − 12 = 83.

DAY 5 · FILTER BY BRAND

```
SELECT name, release_date FROM releases WHERE brand = 'adidas';
```

name	release_date
adidas Anthony Edwards 3 "Purple Iridescent"	2026-11-10
UNheardof x adidas Adistar Control 5 "Doggy Runner"	2026-11-21
Pharrell Williams x adidas Adistar Jellyfish "Dark Brown Frozen Green"	2026-12-05
Pharrell x adidas Adistar Jellyfish "Auburn Timber"	2026-12-05

4 rows

DAY 5 · PYTHON CORNER

B: `False`. <= asks a yes/no question. Is 140 less than or equal to 120? No, so False.

DAY 6 · SEARCH INSIDE NAMES

```
SELECT name, retail_usd FROM releases WHERE name
LIKE '%Space Jam%';
```

name	retail_usd
Jordan Trunner O/S "Space Jam Planet"	130
Air Jordan 1 Mid "Space Jam"	165
Air Jordan 1 Low Golf Spiked "Space Jam"	175
Air Jordan Women's OG "Space Jam Amusement Park"	155

9 rows, first 4 shown

DAY 6 · PYTHON CORNER

B: `True`. in checks if one piece of text is inside another, like LIKE '%Flutter%' in SQL.

DAY 7 · PICK A TIME PERIOD

```
SELECT release_date, name FROM releases WHERE
release_date BETWEEN '2026-11-20' AND '2026-11-30'
ORDER BY release_date;
```

release_date	name
2026-11-20	Nike Book 2 "Be Legendary"
2026-11-21	Air Jordan 3 "Santa"
2026-11-21	Nike Zoom Hyperflight "Safari"
2026-11-21	Victory Lap Radio x Nike Air Force 1 Low

27 rows, first 4 shown

DAY 7 · PYTHON CORNER

C: `Keep saving`. 90 is not >= 110, so Python skips the if part and runs the else part.

DAY 8 · MISSING PRICES

```
SELECT release_date, name FROM releases WHERE
retail_usd IS NULL;
```

release_date	name
2026-11-07	Nike Air Force 1 Low "N7"
2026-11-21	Victory Lap Radio x Nike Air Force 1 Low
2026-11-21	UNheardof x adidas Adistar Control 5 "Doggy Runner"
2026-11-28	Social Status x Air Jordan 1 Low OG

6 rows, first 4 shown

DAY 8 · PYTHON CORNER

D: `Price TBA`. None is Python's NULL. Like SQL, you check it with is None, not == None.

DAY 9 · COUNT AND AVERAGE

```
SELECT COUNT(*) AS releases,
ROUND(AVG(retail_usd)) AS avg_price FROM releases;
```

releases	avg_price
111	185

1 row

DAY 9 · PYTHON CORNER

C: `180.0`. 1 − 0.10 = 0.90, and 200 × 0.90 = 180. The .0 is because 0.10 is a decimal, so the answer is too.

DAY 10 · GROUP BY BRAND

```
SELECT brand, COUNT(*) AS drops,
ROUND(AVG(retail_usd)) AS avg_price FROM releases
GROUP BY brand ORDER BY drops DESC;
```

brand	drops	avg_price
Nike	72	178
Jordan	34	194
adidas	4	243
Vans	1	130

4 rows

DAY 10 · PYTHON CORNER

C: `Flipped the Dash 5 for +$35`. An f in front of the quotes lets you drop variables into text with {curly brackets}.

DAY 11 · DROPS PER WEEK

```
SELECT strftime('%Y-W%W', release_date) AS week,
COUNT(*) AS drops FROM releases GROUP BY week
ORDER BY week;
```

week	drops
2026-W44	34
2026-W45	18
2026-W46	15
2026-W47	14

9 rows, first 4 shown

DAY 11 · PYTHON CORNER

D: `183.7`. round(x, 1) keeps one digit after the decimal point and rounds up from .67.

DAY 12 · BUSY DAYS (HAVING)

```
SELECT release_date, COUNT(*) AS drops FROM
releases GROUP BY release_date HAVING COUNT(*) >=
5 ORDER BY drops DESC;
```

release_date	drops
2026-11-05	23
2026-11-28	11
2026-11-21	11
2026-12-19	8

5 rows, first 4 shown

DAY 12 · PYTHON CORNER

B: `3  5`. // divides and drops the decimals (3 tickets). % gives the remainder (95 − 90 = 5).

DAY 13 · MEET THE SALES LOG

```
SELECT COUNT(*) AS sales FROM sales WHERE
sale_date BETWEEN '2026-09-01' AND '2026-09-30';
```

sales
532

1 row

DAY 13 · PYTHON CORNER

C: `120  150`. Lists count from 0, so [0] is the first item. [-1] means the last item.

DAY 14 · JOIN TWO TABLES

```
SELECT sh.model, sh.colorway, COUNT(*) AS sales,
ROUND(AVG(s.price_usd)) AS avg_price FROM sales s
JOIN shoes sh ON s.shoe_id = sh.shoe_id GROUP BY
sh.shoe_id ORDER BY sales DESC;
```

model	colorway	sales	avg_price
Comet Mid	Gold Rush	185	362
Legacy 85	Varsity Red	155	319
Court King	Bred	153	185
Orbit 3	Cosmic Purple	124	102

20 rows, first 4 shown

DAY 14 · PYTHON CORNER

C: `210.0`. sum is 840, len is 4. 840 ÷ 4 = 210. That's exactly what SQL's AVG() does.

DAY 15 · HEAT OR BRICK? (CASE)

```
SELECT sh.model, sh.colorway, ROUND(100.0 *
(AVG(s.price_usd) - sh.retail_usd) /
sh.retail_usd) AS premium_pct, CASE WHEN
ROUND(100.0 * (AVG(s.price_usd) - sh.retail_usd) /
sh.retail_usd) >= 20 THEN 'HEAT' WHEN ROUND(100.0
* (AVG(s.price_usd) - sh.retail_usd) /
sh.retail_usd) >= 0 THEN 'FAIR' ELSE 'BRICK' END
AS label FROM sales s JOIN shoes sh ON s.shoe_id =
sh.shoe_id GROUP BY sh.shoe_id ORDER BY
premium_pct DESC;
```

model	colorway	premium_pct	label
Comet Mid	Gold Rush	65	HEAT
Legacy 85	Varsity Red	60	HEAT
Court King	Bred	24	HEAT
Dash 5	Shadow	22	HEAT

20 rows, first 4 shown

DAY 15 · PYTHON CORNER

D: `80`. The loop runs once per item: 0 + 20 + 35 + 25 = 80.

DAY 16 · BEAT THE AVERAGE (WITH)

```
WITH per_shoe AS (SELECT shoe_id, AVG(price_usd)
AS avg_price FROM sales GROUP BY shoe_id) SELECT
sh.model, sh.colorway, ROUND(p.avg_price) AS
avg_price FROM per_shoe p JOIN shoes sh ON
sh.shoe_id = p.shoe_id WHERE p.avg_price > (SELECT
AVG(price_usd) FROM sales) ORDER BY p.avg_price
DESC;
```

model	colorway	avg_price
Comet Mid	Gold Rush	362
Legacy 85	Varsity Red	319
Summit Hi	Midnight	190
Court King	Bred	185

4 rows

DAY 16 · PYTHON CORNER

B: `heat 310 then heat 400`. The loop checks every price. Only 310 and 400 pass the > 300 test.

DAY 17 · RANK THE BRANDS

```
SELECT sh.brand, SUM(s.price_usd) AS revenue,
RANK() OVER (ORDER BY SUM(s.price_usd) DESC) AS
place FROM sales s JOIN shoes sh ON s.shoe_id =
sh.shoe_id GROUP BY sh.brand ORDER BY place;
```

brand	revenue	place
Mavrik	103569	1
Halo Athletic	98264	2
Northbound	40445	3
Kitebird	29106	4

4 rows

DAY 17 · PYTHON CORNER

A: 65. 250 × 0.9 = 225, minus 160 = 65. fee=0.10 is a default, used when you don't pass a fee.

DAY 18 · PYTHON: LISTS AND LOOPS

1. C: 130. Positions start at 0, so prices[0] is the first item.

2. A: 3. len() counts the items.

3. D: 600. The loop adds each price to total: 130 + 215 + 255 = 600.

4. A: heat 215, then heat 255. Only prices above 200 pass the if: 215 and 255.

5. A: [20, 40]. A list comprehension builds a new list by applying p * 2 to each item.

DAY 18 · PYTHON CORNER

A: `3 Orbit 3`. .append() adds to the end. Three pairs; [1] is the second one: Orbit 3.

DAY 19 · PYTHON: DICTIONARIES AND FUNCTIONS

1. D: 230. shoe['price'] looks up the value stored under 'price'.

2. A: 70. The function returns 300 - 230 = 70.

3. B: 3. Adding 'size' makes three keys: name, price, size.

4. B: ['Kobe 8']. items() gives name/value pairs; only Kobe 8 has 0 in stock.

5. D: 0.67. round(x, 2) keeps two decimal places, rounding 0.666... up to 0.67.

DAY 19 · PYTHON CORNER

B: `Comet Mid 395`. A dict stores values under names (keys). Setting pair["paid"] replaces the old 410.

DAY 20 · PANDAS: SQL'S TWIN

1. B: releases[['name', 'retail_usd']]. Double brackets with a list of column names picks those columns.

2. D: releases[releases['brand'] == 'Nike']. A True/False mask inside brackets keeps the matching rows.

3. A: releases.sort_values('retail_usd', ascending=False). sort_values with ascending=False is ORDER BY ... DESC.

4. A: releases.groupby('brand').size(). groupby makes the piles; size() counts rows in each pile.

5. D: sales.merge(shoes, on='shoe_id'). merge on a key column is a JOIN. concat just stacks tables.

DAY 20 · PYTHON CORNER

B: `[107, 150, 235]`. A comprehension builds a new list in one line: 100×1.07=107, 140×1.07=149.8→150, 220×1.07=235.4→235.

DAY 21 · HOW SPREAD OUT? (ANALYTICS)

```
SELECT CAST(price_usd / 50 AS INTEGER) * 50 AS
bucket, COUNT(*) AS sales FROM sales GROUP BY
bucket ORDER BY bucket;
```

bucket	sales
0	25
50	613
100	285
150	293

9 rows, first 4 shown

DAY 21 · PYTHON CORNER

C: `370`. 520 − 150 = 370. That $520 sale is an outlier: way off from the rest. Outliers stretch the range a lot.

DAY 22 · CLEAN IT, CHART IT (NOTEBOOK)

1. D: Count them and decide on purpose: drop them or keep them as unknown. Missing isn't zero. Decide deliberately and say what you did.

2. B: So pandas treats the column as dates (weeks, months, sorting) instead of plain text. Real date types unlock .dt.month, .dt.isocalendar() and time charts.

3. A: weeks in time order on the x-axis and a title with the date range. Time goes left to right, and the title says which period.

4. A: Which source, which dates, and when the list was checked. Always name the source and time period.

DAY 22 · PYTHON CORNER

A: `150.0`. groupby + mean is SQL's GROUP BY + AVG(). Kitebird: (140 + 160) ÷ 2 = 150.

DAY 23 · LINE OF BEST FIT (SQL)

```
WITH p AS (SELECT sh.hype AS x, 100.0 *
(AVG(s.price_usd) - sh.retail_usd) / sh.retail_usd
AS y FROM sales s JOIN shoes sh ON s.shoe_id =
sh.shoe_id GROUP BY sh.shoe_id), m AS (SELECT
AVG(x) AS mx, AVG(y) AS my FROM p), f AS (SELECT
SUM((x - mx) * (y - my)) / SUM((x - mx) * (x -
mx)) AS slope, mx, my FROM p, m) SELECT
ROUND(slope, 1) AS slope, ROUND(my - slope * mx,
1) AS intercept FROM f;
```

slope	intercept
9.7	-68.6

1 row

DAY 23 · PYTHON CORNER

B: 9. 9.7 × 8 = 77.6, then 77.6 − 68.6 = 9. About 9% over retail. That's what a model does: plug in x, get a guess for y.

DAY 24 · TEACH A MODEL (GRADIENT DESCENT)

C: The loss blew up: each step overshot the bottom. Too-big steps jump past the bottom and get farther away each time.

DAY 24 · PYTHON CORNER

D: `0.4`. w − lr × gradient = 0 − 0.1 × (−4) = +0.4. The model steps downhill, opposite the slope.

DAY 25 · HEAT DETECTOR (DECISION TREE)

B: The one with the best TEST accuracy. Test accuracy shows how it does on pairs it hasn't seen, which is what happens at the counter.

DAY 25 · PYTHON CORNER

C: `0.6`. zip pairs them up. 3 of 5 match, so 3 ÷ 5 = 0.6, or 60% accuracy.

DAY 26 · CUSTOMER GROUPS (K-MEANS)

B: How many groups (k) to look for. You choose k; the model finds the groups. No labels needed.

DAY 26 · PYTHON CORNER

D: `3.03`. $(-3)^2 + (-0.4)^2 = 9 + 0.16 = 9.16$, and the square root is about 3.03. Notice size dominates, which is why the lab standardizes first.

DAY 27 · YOUR AI TEAMMATE

```
SELECT release_date, name, retail_usd FROM
releases WHERE brand = 'Jordan' AND release_date
BETWEEN '2026-12-01' AND '2026-12-31';
```

release_date	name	retail_usd
2026-12-05	SoleFly x Air Jordan 5	225
2026-12-05	Air Jordan 10 "Sacramento"	215
2026-12-11	JAIDE x Air Jordan 1 Low OG	150
2026-12-12	Air Jordan 11 "Space Jam"	235

8 rows, first 4 shown

DAY 27 · PYTHON CORNER

A: `False`. .count() finds 2, not 3. Always check an AI's numbers against the data.

DAY 28 · WHERE DATA COMES FROM

1. A: So you know what period the numbers cover and how fresh they are. Release dates change. 'Nov 21' means nothing without the year and when someone checked.

2. B: Use an official API or a source that allows it. Follow the rules: official APIs, open data, or sources that permit it.

3. A: Not without permission: they're copyrighted. Link to the article instead. Photos belong to whoever took them. Linking is fine; copying needs permission.

4. D: a password: keep it secret, never in public code. Keys identify you. Leaked keys get abused.

5. A: practice data made up for learning. Practice data. The release calendar is real; the sales log is practice.

DAY 28 · PYTHON CORNER

A: 22. Subtracting dates gives a time gap; .days turns it into a number. Nov 27 − Nov 5 = 22 days.

DAY 29 · PYTHON CORNER

D: 2. Week 1: 425 (not enough). Week 2: 470, enough. A while loop repeats until its test is False.

DAY 30 · GRAIL DAY (FINAL REVIEW)

1. A: SELECT name FROM releases WHERE retail_usd IS NULL;. Missing values are NULL, and you test them with IS NULL. = NULL never matches anything, and 0 is a real price. (Day 8)

2. A: HAVING COUNT(*) >= 5. WHERE filters rows before grouping. HAVING filters the groups after GROUP BY. (Day 12)

3. A: JOIN shoes ON sales.shoe_id = shoes.shoe_id. A JOIN matches rows that share a key. shoe_id is the key both tables have. (Week 3)

4. D: Comet Mid. .append() adds to the end, and [-1] is the last item. (Day 18 Python corner)

5. B: [120, 300]. The comprehension keeps only prices over 100, in their original order. (Day 20 Python corner)

6. B: 0. .get(key, default) gives the default when the key is missing, so no KeyError. (Day 19)

7. A: The median. One huge outlier drags the mean up. The median is the middle value, so one outlier barely moves it. (Day 21)

8. D: It might be no better than always guessing 'brick'. Always compare to the baseline: the score you'd get by guessing the most common answer every time. (Day 25)

9. D: Run a COUNT(*) query yourself and compare. Check an AI's numbers against the data. Asking it again isn't checking. (Day 27)

DAY 30 · PYTHON CORNER

B: `True 552`. Before the fee the pairs total 380. Keeping 90% gives 342. 210 + 342 = 552, and 552 >= 540 is True. Grail secured.



Keep going

Work every shift on the Drop Table page, flip some pairs, survive the curveballs, and get Big Face his grail.

Drop Table is an original course. The Grail Room, Big Face, Rae, Theo and the brands Northbound, Halo Athletic, Kitebird and Mavrik are made up. Release data: Sneaker Files release calendar (https://www.sneakerfiles.com/release-dates/), covering November 5, 2026 to December 31, 2026, checked November 5, 2026. Not affiliated with any brand, store or marketplace.

Created by **Ofgeneration LLC** · © 2026 Ofgeneration LLC · support@grailroom.com